实验目的：

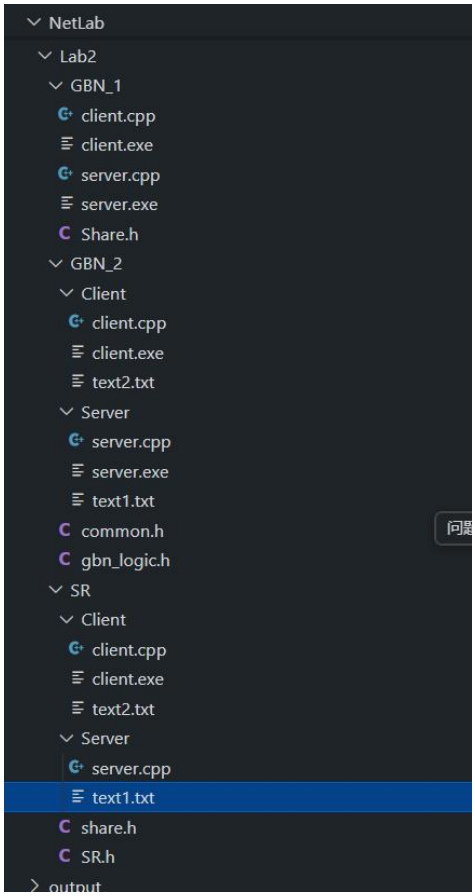
理解可靠数据传输的基本原理；掌握停等协议的工作原理；掌握基于 UDP 设计并实现一个停等协议的过程与技术。

实验内容：

1) 基于 UDP 设计一个简单的停等协议，实现单向可靠数据传输（服务器到客户的数据传输）。
2) 模拟引入数据包的丢失，验证所设计协议的有效性。
3) 改进所设计的停等协议，支持双向数据传输；
4）基于所设计的停等协议，实现一个 C/S 结构的文件传输应用。

1) 基于 UDP 设计一个简单的 GBN 协议，实现单向可靠数据传输（服务器到客户的数据传输）。
2) 模拟引入数据包的丢失，验证所设计协议的有效性。
3) 改进所设计的 GBN 协议，支持双向数据传输；
4）将所设计的 GBN 协议改进为 SR 协议。

实验过程：

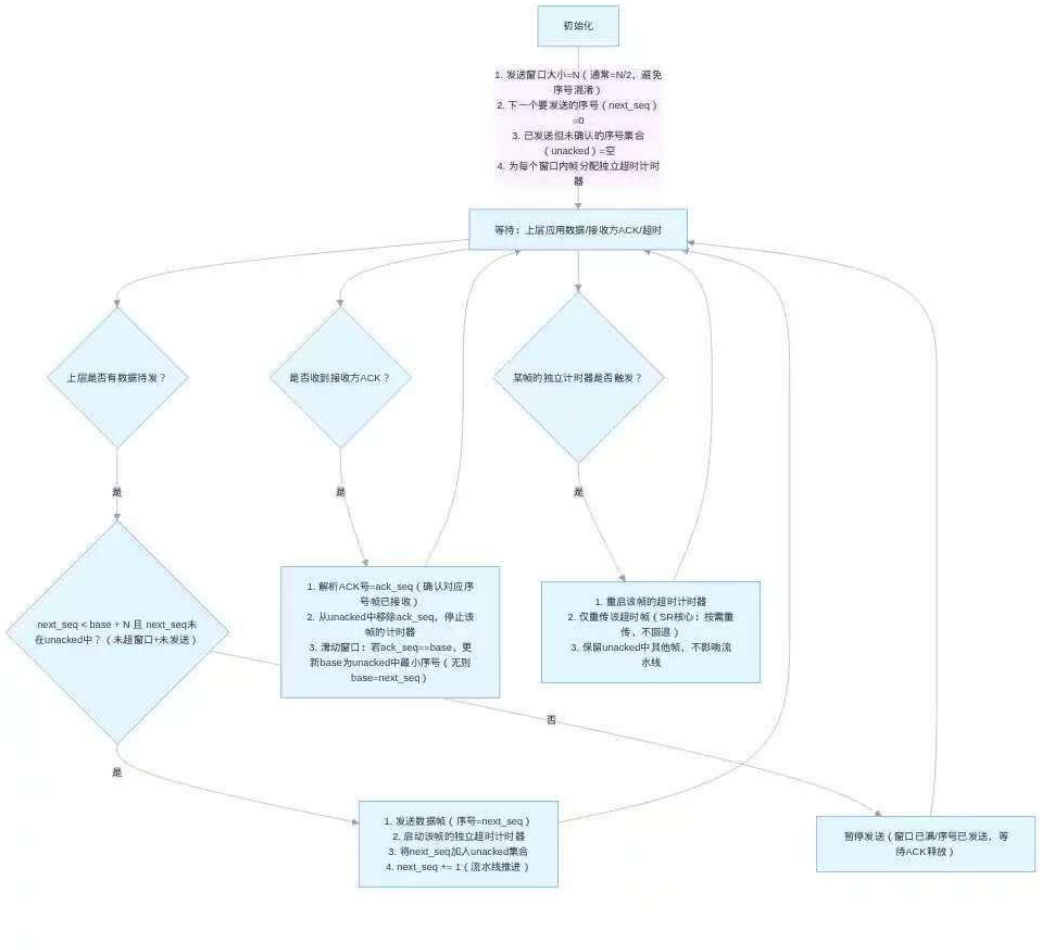


文件结构

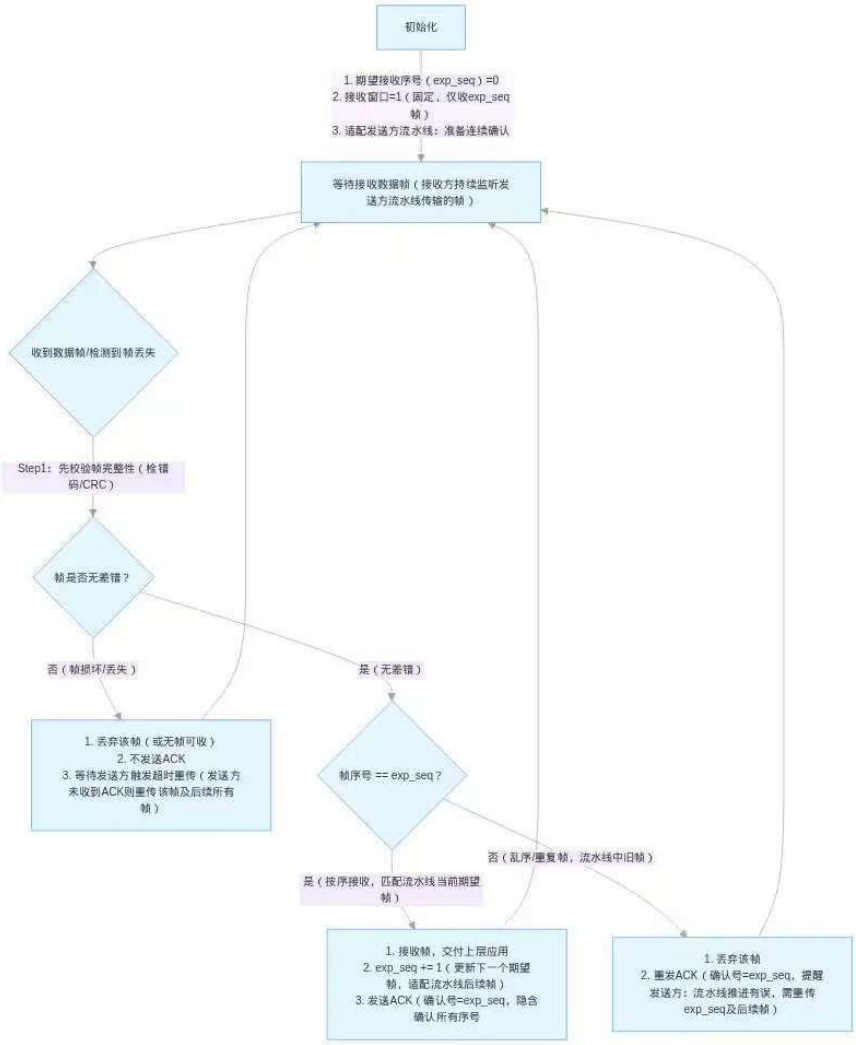
编译指令

g++ server.cpp -o server.exe -std=c++11 -lws2_32

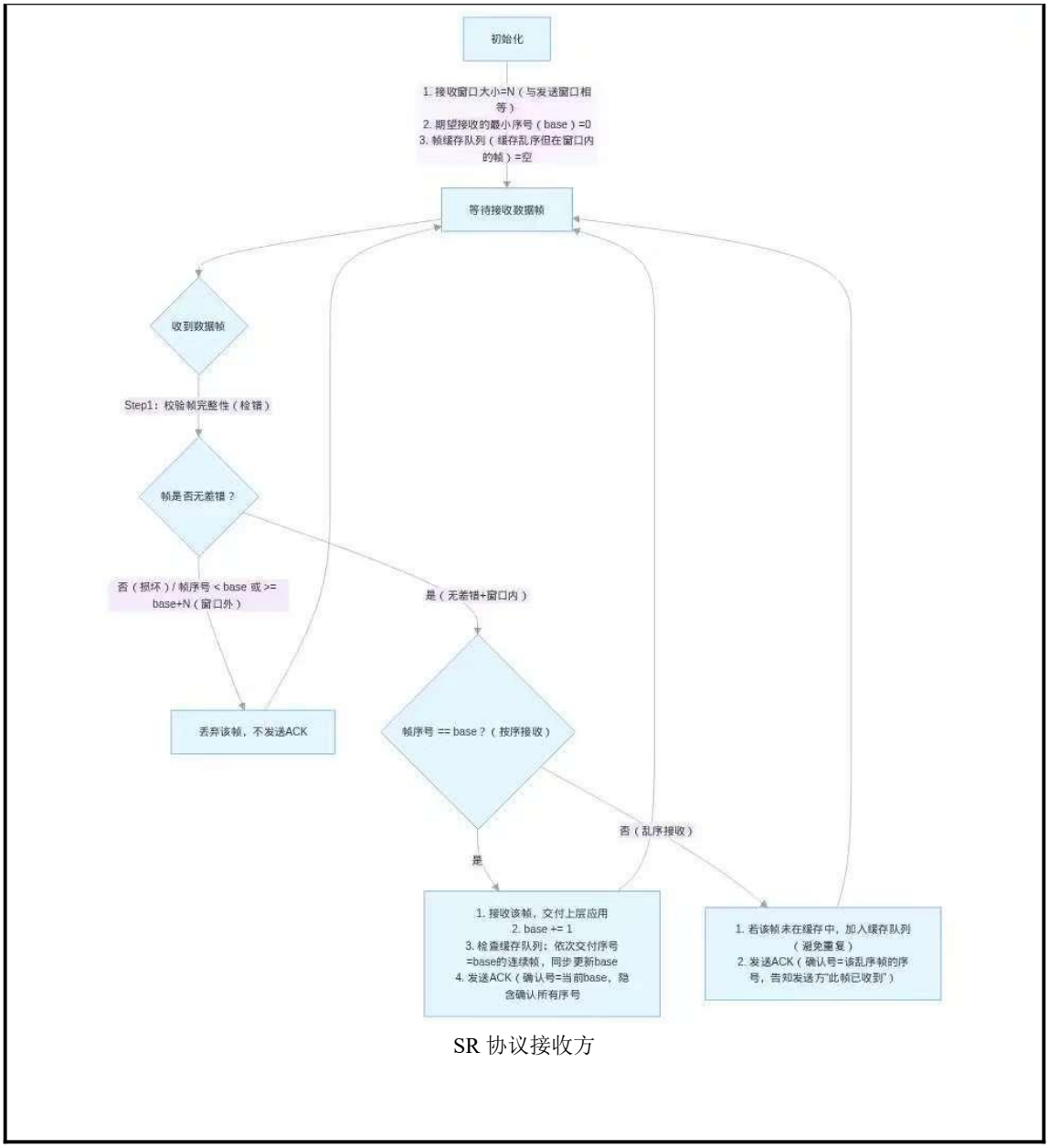
g++ client.cpp -o client.exe -std=c++11 -lws2_32



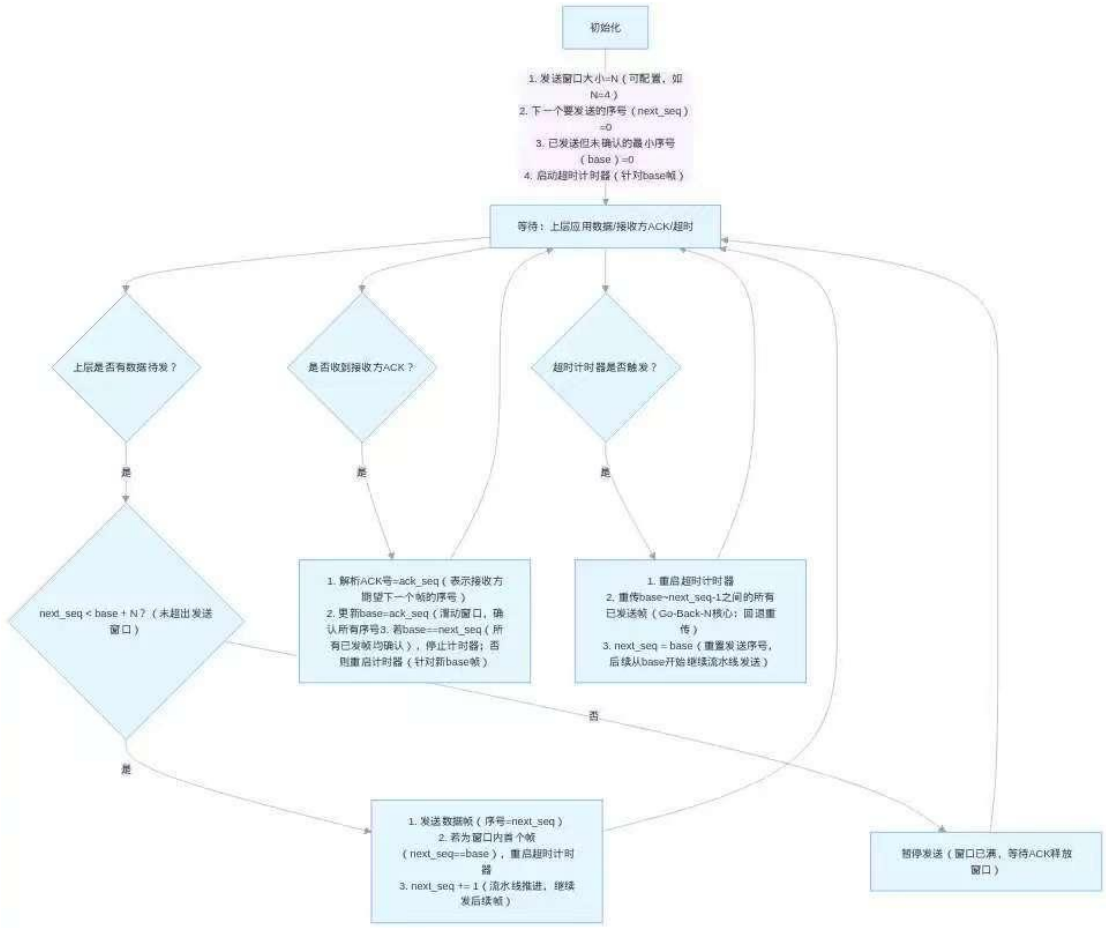
SR 协议发送方


初始化
1. 期望接收序号（exp_seq）=0
2. 接收窗口=1（固定，仅收exp_seq帧）
3. 适配发送方流水线：准备连续确认
等待接收数据帧（接收方持续监听发送方流水线传输的帧）
收到数据帧/检测到帧丢失
Step1：先校验帧完整性（检错码/CRC）
帧是否无差错？
否（帧损坏/丢失）
是（无差错）
1. 丢弃该帧（或无帧可收）
2. 不发送ACK
3. 等待发送方触发超时重传（发送方未收到ACK则重传该帧及后续所有帧）
帧序号 == exp_seq？
否（乱序/重复帧，流水线中旧帧）
是（按序接收，匹配流水线当前期望帧）
1. 接收帧，交付上层应用
2. exp_seq += 1（更新下一个期望帧，适配流水线后续帧）
3. 发送ACK（确认号=exp_seq，隐含确认所有序号
1. 丢弃该帧
2. 重发ACK（确认号=exp_seq，提醒发送方：流水线推进有误，需重传exp_seq及后续帧）

GBN 协议接收方


初始化
1. 接收窗口大小=N（与发送窗口相等）
2. 期望接收的最小序号（base）=0
3. 帧缓存队列（缓存乱序但在窗口内的帧）=空
等待接收数据帧
收到数据帧
Step1：校验帧完整性（检错）
帧是否无差错？
否（损坏）/ 帧序号 < base 或 >= base+N（窗口外）
是（无差错+窗口内）
丢弃该帧，不发送ACK
帧序号 == base？（按序接收）
否（乱序接收）
是
1. 接收该帧，交付上层应用
2. base += 1
3. 检查缓存队列：依次交付序号=base的连续帧，同步更新base
4. 发送ACK（确认号=当前base，隐含确认所有序号
1. 若该帧未在缓存中，加入缓存队列（避免重复）
2. 发送ACK（确认号=该乱序帧的序号，告知发送方"此帧已收到"）

SR 协议接收方



GBN 协议发送方

第一部分：

共享头文件 Share.h

首先，先设计基本的协议报文数据格式。在共同使用的头文件 Share.h 中使用 define 宏定义了服务器端口、载荷大小、超时时间、窗口大小等参数。

```
// 链接 Winsock 库
#pragma comment(lib, "ws2_32.lib")

// 协议配置
#define SERVER_PORT 8888        // 服务器端口
#define TIMEOUT_MS 2000         // GBN 超时时间 (2000ms)
// (L=8 (1 字节 Seq), W+1 <= 2^L -> W+1 <= 256 -> W <= 255)
#define WINDOW_SIZE 15          // GBN 窗口大小 (W)
// (Data <= 1024)
#define DATA_PAYLOAD_SIZE 1024 // 数据帧中数据 payload 的最大大小
// (Seq(1) + Data(1024) + EOF(1))
```

```
#define FRAME_HEADER_SIZE 1    // 1 字节 序列号 (Seq)
#define FRAME_TRAILER_SIZE 1   // 1 字节 结束标记 (EOF Flag)
#define MAX_FRAME_SIZE (FRAME_HEADER_SIZE + DATA_PAYLOAD_SIZE +
FRAME_TRAILER_SIZE) // 1026 字节
// (ACK 1 字节)
#define ACK_FRAME_SIZE 1       // 1 字节 ACK
// (EOF Flag. 1=Data, 0=End)
#define FLAG_DATA 1
#define FLAG_EOF 0
// 模拟发送的数据包总数 (用于 -testgbn)
#define TOTAL_PACKETS 30
// 日志记录辅助函数 (不变)
static std::mutex logMutex;
```

为了便于打印日志和输出当前时间，编写对应函数。

```
// 获取当前时间字符串 H:M:S.ms
static std::string getCurrentTime() {
    auto now = std::chrono::system_clock::now();
    auto in_time_t = std::chrono::system_clock::to_time_t(now);
    auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_ep
och()) % 1000;
    std::tm bt;
    localtime_s(&bt, &in_time_t); // Windows-specific-safe afunction
    std::ostringstream ss;
    ss << std::put_time(&bt, "%H:%M:%S");
    ss << '.' << std::setfill('0') << std::setw(3) << ms.count();
    return ss.str();
}

// 打印日志
static void log(const std::string& prefix, const std::string& message) {
    std::lock_guard<std::mutex> lock(logMutex);
    std::cout << "[" << getCurrentTime() << "] " << prefix << ": " << message
<< std::endl;
}
```

上方 log()函数为日志打印函数。

将超时设置打包为函数。

```
// 辅助函数：设置 socket 超时
static void setSocketTimeout(SOCKET sock, DWORD timeout_ms) {
    if (setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout_ms,
sizeof(timeout_ms)) == SOCKET_ERROR) {
        log("NET_ERROR", "setsockopt (SO_RCVTIMEO) failed: " +
```

```
std::to_string(WSAGetLastError()));
    }
}
```

server.cpp

在服务器函数中，先在 main()函数中初始化参数并绑定端口。

```
WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        log("SERVER_ERROR", "WSAStartup failed.");
        return 1;
    }

    SOCKET serverSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (serverSocket == INVALID_SOCKET) {
        log("SERVER_ERROR", "Socket creation failed.");
        WSACleanup();
        return 1;
    }
    // 默认使用阻塞模式 (timeout=0) 来等待命令
    setSocketTimeout(serverSocket, 0);
    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(SERVER_PORT);
    if (bind(serverSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
SOCKET_ERROR) {
        log("SERVER_ERROR", "Bind failed.");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }
    log("SERVER", "Server is running, waiting for commands on port " +
std::to_string(SERVER_PORT));
```

初始化参数完毕后，使用阻塞模式下的 recvfrom()函数，等待命令。

```
    while (true) {
        int recvLen = recvfrom(serverSocket, cmdBuffer, sizeof(cmdBuffer)
- 1, 0, (SOCKADDR*)&clientAddr, &clientAddrLen);
        if (recvLen <= 0) {
            log("SERVER_ERROR", "recvfrom (command) failed.");
            continue;
        }

        cmdBuffer[recvLen] = '\0'; // 确保字符串结束
```

```
        std::string command(cmdBuffer);
        log("SERVER", "Received command: '" + command + "'");
        std::string response;
```

不断循环接收命令。根据客户端发送的命令进行相应的处理。

```
        if (command == "-time") {
            response = "Server time: " + getCurrentTime();
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
        }
```

如果是-time 指令，直接利用在 Share.h 中的参数获取时间并构建时间字符串发送。

```
        else if (command == "-quit") {
            response = "Good bye!";
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
        }
```

如果是-quit 指令，向客户端发送“Good bye！”。

```
        else if (command.rfind("-testgbn", 0) == 0) {
            response = "ACK: Starting GBN test.";
            // 回复确认启动 GBN
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
            // 进入 GBN 发送状态
            startGbnSend(serverSocket, clientAddr);
        }
```

如果是-testgbn 指令，则向客户端发送确认信息 GBN，并调用 startGbnSend()函数开始发送模拟。

```
        else {
            // Echo
            response = "ECHO: " + command;
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
        }
    }
```

否则不是现有的命令，将收到的命令重新发送回去。

接下来介绍关键函数 startGbnSend()

```
   log("SERVER", "Entering GBN transfer state...");
    prepareFrames(); // 准备所有数据帧
```

首先打印日志信息，并调用 prepareFrames()函数准备所有数据帧。

```
void prepareFrames() {
```

```
    allFrames.clear();
    log("SERVER_GBN", "Preparing " + std::to_string(TOTAL_PACKETS) + "
frames...");

    for (int i = 0; i < TOTAL_PACKETS; ++i) {
        // 创建一个 1026 字节的缓冲区
        std::vector<char> frame(MAX_FRAME_SIZE, 0); // 初始化为 0
        // 序列号 (1 字节)
        frame[0] = static_cast<uint8_t>(i);

        // 数据 (<= 1024 字节)
        std::string data = "This is packet " + std::to_string(i);
        memcpy(&frame[FRAME_HEADER_SIZE], data.c_str(), data.length());
        // 最后一个字节 (EOF Flag)
        // 1 表示数据, 0 表示结束
        frame[MAX_FRAME_SIZE - 1] = (i == TOTAL_PACKETS - 1) ? FLAG_EOF :
FLAG_DATA;
        allFrames.push_back(frame);
    }
}
```

数据帧采用 1026 字节的固定长度，包含 1 字节序列号、1024 字节数据载荷和 1 字节 EOF 标志；ACK 帧简化为仅包含 1 字节确认序列号。

继续查看 startGbnsend()函数

```
    // GBN 状态变量 (使用 1 字节序列号)
    uint8_t base = 0;
    uint8_t nextSeqNum = 0;

    // 切换到非阻塞 (1ms 轮询) 模式
    setSocketTimeout(serverSocket, 1);

    // GBN 唯一计时器
    auto gbn_timer_start = std::chrono::steady_clock::now();
    bool is_timer_running = false;
```

初始化 GBN 协议需要的一些基本变量，比如最早发送的未确认的包序号 base、下一个要发送的包 nextSeqNum 以及计时器。

```
    // ACK 缓冲区
    char ackBuffer[ACK_FRAME_SIZE];
    sockaddr_in fromAddr;
    int fromAddrLen = sizeof(fromAddr);
```

定义 ACK 缓冲区。

```
    // 只要 base 还没追上所有包的总数，就继续循环
    while (base < TOTAL_PACKETS) {
```

```
        // (1) [发送] 检查窗口是否已满，如果不满，立即发送新包
        // (W=15)
        while (nextSeqNum < base + WINDOW_SIZE && nextSeqNum < TOTAL_PACKETS)
{
            const auto& frameToSend = allFrames[nextSeqNum];

            log("SERVER_GBN", "SEND Data " + std::to_string(nextSeqNum));
            sendto(serverSocket, frameToSend.data(), MAX_FRAME_SIZE, 0,
(SOCKADDR*)&clientAddr, sizeof(clientAddr));

            if (base == nextSeqNum) {
                gbn_timer_start = std::chrono::steady_clock::now();
                is_timer_running = true;
                log("SERVER_GBN", "TIMER START for base " +
std::to_string(base));
            }
            nextSeqNum++;
        }
```

服务器端采用流水线发送机制，维护一个大小为 15 的滑动窗口。如果 base ==nextSeqNum，启动单一计时器监控最早未确认包的超时情况。当窗口未满时立即发送新数据包，并使 nextSeqNum + 1。

```
        // (2) [接收] 非阻塞地检查是否有 1 字节的 ACK 到达
        int recvLen = recvfrom(serverSocket, ackBuffer, ACK_FRAME_SIZE, 0,
(SOCKADDR*)&fromAddr, &fromAddrLen);
```

根据接受数据大小进行判断，如果接收到 ACK 数据包。分类讨论情况，如果是未确认的序列号，就令 base 移动到确认的序列号+1 的位置(GBN 协议采用累计确认，一旦收到一个位置的 ACK 就说明该序列号之前的数据包已经被全部接收)，并重启计时器。如果 base == nextSeqNum，就说明窗口的起点已经移动到下一个待发送的位置，需要停止计时。对于客户端发来的命令则选择丢弃。

```
        if (recvLen == ACK_FRAME_SIZE) {
            // (1 字节 ACK)
            uint8_t ackNum = static_cast<uint8_t>(ackBuffer[0]);
            log("SERVER_GBN", "RECV ACK " + std::to_string(ackNum));

            // 累积确认: 如果收到的 ACK >= base，就移动窗口
            if (ackNum >= base) {
                base = ackNum + 1;
                log("SERVER_GBN", "Window base moved to " +
std::to_string(base));

                if (base == nextSeqNum) {
```

```
                    is_timer_running = false;
                    log("SERVER_GBN", "TIMER STOP (window empty)");
                } else {
                    gbn_timer_start = std::chrono::steady_clock::now();
                    log("SERVER_GBN", "TIMER RESTART for new base " +
std::to_string(base));
                }
            }
        } else if (recvLen > 0) {
            log("SERVER_WARN", "Received an invalid ACK frame size: " +
std::to_string(recvLen));
            // 可能是客户端发来的命令，丢弃
        } else {
            int error = WSAGetLastError();
            if (error != WSAETIMEDOUT && error != 0) {
                log("SERVER_ERROR", "recvfrom poll failed: " +
std::to_string(error));
            }
        }
```

检查是否超时， 如果超时则重传窗口内已发送的数据包，并重启计时器。

```
        // (3) [计时] 检查 GBN 的唯一计时器是否超时
        if (is_timer_running) {
            auto now = std::chrono::steady_clock::now();
            auto duration =
std::chrono::duration_cast<std::chrono::milliseconds>(now -
gbn_timer_start).count();
            if (duration > TIMEOUT_MS) {
                log("SERVER_GBN", "TIMEOUT! Retransmitting window from " +
std::to_string(base));

                for (uint8_t i = base; i < nextSeqNum; ++i) {
                    const auto& frameToResend = allFrames[i];
                    log("SERVER_GBN", "RESEND Data " + std::to_string(i));
                    sendto(serverSocket, frameToResend.data(),
MAX_FRAME_SIZE, 0, (SOCKADDR*)&clientAddr, sizeof(clientAddr));
                }

                // 重启计时器
                gbn_timer_start = std::chrono::steady_clock::now();
                log("SERVER_GBN", "TIMER RESTART after retransmission");
            }
        }
```

```
        Sleep(1); // 避免 CPU 100% 占用
    }
```

全部发送完毕之后，切换回阻塞模式，准备等待命令。

```
    log("SERVER", "GBN transfer complete. Returning to command mode.");
    // 切换回阻塞模式，等待新命令
    setSocketTimeout(serverSocket, 0);
```

client.cpp

无论是哪一部分的内容，都是通过在 client.cpp 中模拟 ACK 丢失或者模拟丢包的。

main()函数中，首先先进行套接字初始化设置。

```
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        log("CLIENT_ERROR", "WSAStartup failed.");
        return 1;
    }

    SOCKET clientSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (clientSocket == INVALID_SOCKET) {
        log("CLIENT_ERROR", "Socket creation failed.");
        WSACleanup();
        return 1;
    }
    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr); // 连接本地服务
器
    // 初始化随机数种子，用于模拟丢包
    srand(static_cast<unsigned int>(time(NULL)));
```

进入无限循环，不断接收终端中输入的命令行指令并发送给服务器端。

```
while (true) {
        std::cout << "\n> ";
        std::getline(std::cin, input);
        if (input.empty()) continue;

        // 发送命令
        sendto(clientSocket, input.c_str(), input.length(), 0,
(SOCKADDR*)&serverAddr, sizeof(serverAddr));
```

对于-quit 指令，选择等待回复

```
        if (input == "-quit") {
            log("CLIENT", "Quitting...");
```

```
            // (等待 "Good bye!" 回复)
        }
```

```
        // 等待服务器回复
        sockaddr_in fromAddr;
        int fromAddrLen = sizeof(fromAddr);
        int recvLen = recvfrom(clientSocket, responseBuffer,
sizeof(responseBuffer) - 1, 0, (SOCKADDR*)&fromAddr, &fromAddrLen);
```

```
        if (recvLen == SOCKET_ERROR) {
            log("CLIENT_ERROR", "Server timed out.");
            continue;
        }

        responseBuffer[recvLen] = '\0';
        std::string response(responseBuffer);
        log("SERVER_REPLY", response);
```

如果收到并打印出来自服务器的“Good bye!”的话就自动退出客户端。

```
        if (input == "-quit") {
            break; // 收到 "Good bye!" 后退出
        }
```

如果是-testgbn 指令的话，解析设定的丢包率和 ACK 丢失率，并设置，随后进入 GBN 接收状态。

```
        // (-testgbn [X] [Y])
        if (input.rfind("-testgbn", 0) == 0) {
            // 解析 X 和 Y
            std::stringstream ss(input);
            std::string cmd, x_str, y_str;
            ss >> cmd >> x_str >> y_str;
            dataLossRate = x_str.empty() ? 0.2 : std::stod(x_str);
            ackLossRate = y_str.empty() ? 0.2 : std::stod(y_str);

            // 进入 GBN 接收状态
            startGbnReceive(clientSocket, serverAddr);
        }
    }
```

```
void startGbnReceive(SOCKET clientSocket, const sockaddr_in& serverAddr)
{
    log("CLIENT", "Entering GBN receive state...");
    log("CLIENT", "Data Loss Rate (X): " + std::to_string(dataLossRate) +
", ACK Loss Rate (Y): " + std::to_string(ackLossRate));
```

```
    // 接收缓冲区
    char recvBuffer[MAX_FRAME_SIZE];
    uint8_t expectedSeqNum = 0;

    // GBN 接收需要超时，以防服务器崩溃
    setSocketTimeout(clientSocket, TIMEOUT_MS * 2);
```

设置 GBN 接收的超时时间。

```
    while (true) {
        sockaddr_in fromAddr;
        int fromAddrLen = sizeof(fromAddr);
        int recvLen = recvfrom(clientSocket, recvBuffer, MAX_FRAME_SIZE, 0,
(SOCKADDR*)&fromAddr, &fromAddrLen);
        if (recvLen == SOCKET_ERROR) {
            log("CLIENT_ERROR", "recvfrom (GBN) timed out. Exiting GBN
mode.");
            break;
        }
```

初始化参数设置。

```
        if (recvLen != MAX_FRAME_SIZE) {
            log("CLIENT_WARN", "Received incomplete frame, size: " +
std::to_string(recvLen) + ". Discarding.");
            continue;
        }
```

没有达到最大数据帧长度就继续接收消息。

```
        // (模拟数据包丢失 [X])
        if ((static_cast<double>(rand()) / RAND_MAX) < dataLossRate) {
            log("CLIENT_GBN", "SIMULATE DATA LOSS of Seq " +
std::to_string(static_cast<uint8_t>(recvBuffer[0])));
            continue; // 丢弃该包，不发送 ACK
        }
```

根据随机数测试是否丢弃数据包。

```
        // 解析数据帧
        uint8_t seqNum = static_cast<uint8_t>(recvBuffer[0]);
        bool isEof = (recvBuffer[MAX_FRAME_SIZE - 1] == FLAG_EOF);
```

解析数据包中的信息(序列号、数据/结束)

```
        log("CLIENT_GBN", "RECV Data " + std::to_string(seqNum));
        if (seqNum == expectedSeqNum) {
            // (1) 期望的包
            log("CLIENT_GBN", "ACCEPT Data " + std::to_string(seqNum));

            // "将数据交给上层" (我们只打印)
            // std::string data(&recvBuffer[FRAME_HEADER_SIZE],
```

```
DATA_PAYLOAD_SIZE);
            // log("CLIENT_DATA", data.c_str());
            // 发送对该包的 ACK
            sendAck(clientSocket, serverAddr, expectedSeqNum);
            // 期望的序列号+1 (由于 TOTAL_PACKETS < 255, 不处理回绕)
            expectedSeqNum++;
```

如果是结束的包，就结束接收状态，退出循环。

```
            if (isEof) {
                log("CLIENT", "Received EOF. GBN transfer complete.");
                break; // 结束 GBN 接收
            }
        } else {
            // (2) GBN 特性：失序的包
            log("CLIENT_GBN", "DISCARD (Out of Order) Data " +
std::to_string(seqNum) + ". Expected: " + std::to_string(expectedSeqNum));
            // 重新发送上一个成功接收的 ACK
            if (expectedSeqNum > 0) {
                sendAck(clientSocket, serverAddr, expectedSeqNum - 1);
            }
            // (如果 expectedSeqNum=0, 还没收到过, 不发 ACK)
        }
    }
```

对于失序的包，选择丢弃并重新发送上一个 ACK。

```
    // GBN 结束后，恢复阻塞等待 (用于接收命令回复)
    setSocketTimeout(clientSocket, TIMEOUT_MS);
}
```

ACK 发送函数 sendAck()

```
void sendAck(SOCKET clientSocket, const sockaddr_in& serverAddr, uint8_t
ackNum) {
    // (模拟 ACK 丢失 [Y])
    if ((static_cast<double>(rand()) / RAND_MAX) < ackLossRate) {
        log("CLIENT_GBN", "SIMULATE ACK LOSS for " +
std::to_string(ackNum));
        return; // ACK 丢失
    }

    // ( ACK 帧为 1 字节)
    char ackFrame = static_cast<char>(ackNum);

    log("CLIENT_GBN", "SEND ACK " + std::to_string(ackNum));
    if (sendto(clientSocket, &ackFrame, ACK_FRAME_SIZE, 0,
```

```
(SOCKADDR*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        log("CLIENT_ERROR", "sendto (ACK) failed: " +
std::to_string(WSAGetLastError()));
    }
}
```

利用它来实现随机模拟 ACK 丢包。

第二部分

common.h

其实与第一部分的 Share.h 内容基本相同。主要添加了与校验和有关的机制。

```
// 计算 16 位互联网校验和 (字节序安全)
static uint16_t calculateChecksum(const char* data, int len) {
    uint32_t sum = 0;
    const uint16_t* ptr = reinterpret_cast<const uint16_t*>(data);

    // 每次加两个字节
    while (len > 1) {
        sum += *ptr++;
        if (sum & 0xFFFF0000)
            sum = (sum & 0xFFFF) + (sum >> 16);
        len -= 2;
    }
    // 如果是奇数个字节，补 0
    if (len > 0) {
        uint16_t last = static_cast<unsigned char>(*reinterpret_cast<const
unsigned char*>(ptr));
        sum += last;
        if (sum & 0xFFFF0000)
            sum = (sum & 0xFFFF) + (sum >> 16);
    }
    // 取反码
    return static_cast<uint16_t>(~sum);
}
```

```
// 验证校验和
static bool verifyChecksum(const char* buffer, int len) {
    if (len < 8) return false; // UDP 头部最少 8 字节
    // 取出原始校验和字段（位于偏移 6 字节处）
    uint16_t received_checksum;
    std::memcpy(&received_checksum, buffer + 6, sizeof(uint16_t));
    // 复制数据报并将校验和字段清零
```

```
    char* temp = new char[len];
    std::memcpy(temp, buffer, len);
    std::memset(temp + 6, 0, 2);  // UDP 头部 checksum 字段清 0
    // 计算新的校验和
    uint16_t computed = calculateChecksum(temp, len);
    delete[] temp;
    // 若相等则正确
    return (computed == received_checksum);
}
```

```
// 辅助函数：在数据帧中设置校验和
static void setChecksum(std::vector<char>& frame) {
    frame[FRAME_HEADER_SIZE] = 0;
    frame[FRAME_HEADER_SIZE + 1] = 0;
    uint16_t checksum = calculateChecksum(frame.data(), MAX_FRAME_SIZE);
    frame[FRAME_HEADER_SIZE] = (checksum >> 8) & 0xFF;
    frame[FRAME_HEADER_SIZE + 1] = checksum & 0xFF;
}
```

gbn_logic.h

这个是主要的逻辑文件，由于支持文件上传和下载后，服务器端与客户端在网络上进行的操作没有本质区别，因此，把实现收发文件的函数提取出来成为共有的头文件，从而简化了代码结构。

文件发送函数

```
bool gbnSendFile(SOCKET sock, const sockaddr_in& clientAddr, const
std::string& filename, const std::string& logPrefix)
```

```
    //  (打开文件)
    std::ifstream inFile(filename, std::ios::binary);
    if (!inFile) {  return false; }

    // 1. 加载所有帧
    std::vector<std::vector<char>> allFrames;
    uint8_t seq = 0;
    while (true) {
        std::vector<char> frame(MAX_FRAME_SIZE, 0); // 1028 字节
```

为了提高传输效率，打开文件并生成数据帧后，会将数据帧压入 vector 中，便于从内存中取出。初始化变量。

```
        // 1. Seq
        frame[0] = seq;
        // 2. 校验和 (暂时留空 0)
        // 3. Data (注意偏移量)
        inFile.read(&frame[FRAME_HEADER_SIZE + FRAME_CHECKSUM_SIZE],
```

```
DATA_PAYLOAD_SIZE);
        int readCount = (int)inFile.gcount();
        if (readCount == 0 && allFrames.empty()) break;
```

文件为空(加载失败时)退出

```
        // 4. Flag
        frame[MAX_FRAME_SIZE - 1] = (inFile.eof() || readCount <
DATA_PAYLOAD_SIZE) ? FLAG_EOF : FLAG_DATA;

        // 5. [新增] 计算并设置校验和
        setChecksum(frame); // 调用 common.h 中的辅助函数
```

为数据帧添加校验和

```
        allFrames.push_back(frame);
        seq++;
        if (frame[MAX_FRAME_SIZE - 1] == FLAG_EOF) break;
    }
    inFile.close();
```

如果为最后结束发送的帧，退出循环(需要发送的内容已全部载入)

```
    if (allFrames.empty()) {
        log(logPrefix, "No data to send.");
        return true;
    }

    // 2. GBN 发送逻辑
    //  (base, nextSeqNum, 1ms 轮询, timer  )
    // (GBN 发送循环)
    uint8_t base = 0;
    uint8_t nextSeqNum = 0;
    setSocketTimeout(sock, 1); // 1ms 轮询
    auto gbn_timer_start = std::chrono::steady_clock::now();
    bool is_timer_running = false;
    char ackBuffer[ACK_FRAME_SIZE];
    sockaddr_in fromAddr;
    int fromAddrLen = sizeof(fromAddr);
```

仿照之前实现的发送逻辑。

```
    while (base < allFrames.size()) {
        // (1) 发送
        while (nextSeqNum < allFrames.size()) {
            uint8_t distance = (nextSeqNum - base + 256) % 256;
            if (distance >= WINDOW_SIZE) break;
            const auto& frameToSend = allFrames[nextSeqNum];
            log(logPrefix, "SEND Data " + std::to_string(nextSeqNum));
            sendto(sock, frameToSend.data(), MAX_FRAME_SIZE, 0,
(SOCKADDR*)&clientAddr, sizeof(clientAddr));
```

```
            if (base == nextSeqNum) {
                gbn_timer_start = std::chrono::steady_clock::now();
                is_timer_running = true;
                log(logPrefix, "TIMER START for base " +
std::to_string(base));
            }
            nextSeqNum++;
        }
```

接收 ACK 也是同理。

```
        // (2) 接收 ACK
        int recvLen = recvfrom(sock, ackBuffer, ACK_FRAME_SIZE, 0,
(SOCKADDR*)&fromAddr, &fromAddrLen);
        if (recvLen == ACK_FRAME_SIZE) {
            uint8_t ackNum = static_cast<uint8_t>(ackBuffer[0]);
            log(logPrefix, "RECV ACK " + std::to_string(ackNum));

            uint8_t distance = (ackNum - base + 256) % 256;
            if (distance < WINDOW_SIZE) {
                base = ackNum + 1;
                log(logPrefix, "Window base moved to " +
std::to_string(base));
                if (base == nextSeqNum) {
                    is_timer_running = false;
                    log(logPrefix, "TIMER STOP (window empty)");
                } else {
                    gbn_timer_start = std::chrono::steady_clock::now();
                    log(logPrefix, "TIMER RESTART for new base " +
std::to_string(base));
                }
            }
        } else { /* 1ms 轮询超时，不做处理，等待下一轮 */ }
```

```
        // (3) 检查超时
        if (is_timer_running) {
            auto duration =
std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::stea
dy_clock::now() - gbn_timer_start).count();
            if (duration > TIMEOUT_MS) {
                log(logPrefix, "TIMEOUT! Retransmitting window from " +
std::to_string(base));
                for (uint8_t i = base; i != nextSeqNum; ++i) {
                    const auto& frameToResend = allFrames[i];
                    log(logPrefix, "RESEND Data " + std::to_string(i));
                    sendto(sock, frameToResend.data(), MAX_FRAME_SIZE, 0,
```

```
(SOCKADDR*)&clientAddr, sizeof(clientAddr));
                }
                gbn_timer_start = std::chrono::steady_clock::now();
                log(logPrefix, "TIMER RESTART after retransmission");
            }
        }
        Sleep(1);
    }
    //  (结束)
    log(logPrefix, "GBN send complete.");
    setSocketTimeout(sock, 0); // 恢复阻塞
    return true;
}
```

以及接收文件的函数

```
bool gbnReceiveFile(SOCKET sock, const sockaddr_in& serverAddr, const
std::string& filename,
                    const std::string& logPrefix, double dataLossRate,
double ackLossRate)
```

打开要写入的文件。

```
 std::ofstream outFile(filename, std::ios::binary);
    if (!outFile) {  return false; } // 文件打开失败

    char recvBuffer[MAX_FRAME_SIZE]; // 1028 字节
    uint8_t expectedSeqNum = 0;
    setSocketTimeout(sock, TIMEOUT_MS * 2);
```

设置超时时间。

```
    while (true) {
        int recvLen = recvfrom(sock, recvBuffer, MAX_FRAME_SIZE, 0, NULL,
NULL);
        if (recvLen == SOCKET_ERROR) { break; }
```

先检查接收到的数据帧长度，说明非数据帧类型，重新接收

```
        // (1) 检查长度 (现在必须是 1028)
        if (recvLen != MAX_FRAME_SIZE) {
            log(logPrefix + "_WARN", "Received incomplete frame, size: " +
std::to_string(recvLen));
            continue;
        }

        // (2) [新增] 验证校验和
        // (必须在模拟丢包之前，因为损坏的包也应该被丢弃)
        if (!verifyChecksum(recvBuffer, MAX_FRAME_SIZE)) {
            log(logPrefix, "DISCARD (Checksum Error) Data " +
std::to_string(static_cast<uint8_t>(recvBuffer[0])));
```

```
            // 丢弃损坏的帧，不发送 ACK
            continue;
        }
```

检查校验和是否正确，若损坏，直接丢弃。

```
        // (3) 模拟数据包丢失 [X]
        if ((static_cast<double>(rand()) / RAND_MAX) < dataLossRate) {
            log(logPrefix, "SIMULATE DATA LOSS of Seq " +
std::to_string(static_cast<uint8_t>(recvBuffer[0])));
            continue;
        }
```

```
        // (4) 解析 (校验和通过后)
        uint8_t seqNum = static_cast<uint8_t>(recvBuffer[0]);
        bool isEof = (recvBuffer[MAX_FRAME_SIZE - 1] == FLAG_EOF);
        log(logPrefix, "RECV Data " + std::to_string(seqNum));

        if (seqNum == expectedSeqNum) {
            log(logPrefix, "ACCEPT Data " + std::to_string(seqNum));

            if (!isEof) {
                // (数据偏移量已更改，跳过 Seq(1) 和 Checksum(2))
                outFile.write(&recvBuffer[FRAME_HEADER_SIZE +
FRAME_CHECKSUM_SIZE], DATA_PAYLOAD_SIZE);
            }
```

如果不是文件结束帧，说明为普通数据帧，写入打开的文件。

```
            // (5) 模拟 ACK 丢失 [Y]
            if ((static_cast<double>(rand()) / RAND_MAX) < ackLossRate) {
                log(logPrefix, "SIMULATE ACK LOSS for " +
std::to_string(expectedSeqNum));
            } else {
                char ackFrame = static_cast<char>(expectedSeqNum);
                log(logPrefix, "SEND ACK " +
std::to_string(expectedSeqNum));
                sendto(sock, &ackFrame, ACK_FRAME_SIZE, 0,
(SOCKADDR*)&serverAddr, sizeof(serverAddr));
            }
```

如果产生的随机数小于 ACK 丢失率，就模拟丢掉 ACK，否则发送对应序列号的 ACK。

```
            expectedSeqNum++;
            if (isEof) {  break; } // 结束接收
```

如果是结束帧，就退出循环，结束接受。

```
        } else {
            // (6) 失序包 (GBN)
            log(logPrefix, "DISCARD (Out of Order) Data " +
std::to_string(seqNum) + ". Expected: " + std::to_string(expectedSeqNum));
```

```
            if (expectedSeqNum > 0) {
                // (模拟 ACK 丢失 [Y])
                if ((static_cast<double>(rand()) / RAND_MAX) < ackLossRate)
{
                    log(logPrefix, "SIMULATE ACK LOSS for (Dup) " +
std::to_string(expectedSeqNum - 1));
                } else {
                    char ackFrame = static_cast<char>(expectedSeqNum - 1);
                    log(logPrefix, "RESEND ACK " +
std::to_string(expectedSeqNum - 1));
                    sendto(sock, &ackFrame, ACK_FRAME_SIZE, 0,
(SOCKADDR*)&serverAddr, sizeof(serverAddr));
                }
            }
        }
    }
    outFile.close();
    return true;
```

client.cpp

```
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        log("CLIENT_ERROR", "WSAStartup failed."); return 1;
    }
    SOCKET clientSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    setSocketTimeout(clientSocket, TIMEOUT_MS); // 默认超时

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);
    srand(static_cast<unsigned int>(time(NULL)));
```

初始化套接字与随机数。

```
    std::string input;
    char responseBuffer[1024];
    while (true) {
        std::cout << "\n> ";
        std::getline(std::cin, input);
        if (input.empty()) continue;
        // 解析命令
        std::string cmd, arg1, arg2;
        std::stringstream ss(input);
        ss >> cmd >> arg1 >> arg2;
```

```
        // (1) 特殊本地命令 setloss
        if (cmd == "-setloss") {
            dataLossRate = arg1.empty() ? 0.2 : std::stod(arg1);
            ackLossRate = arg2.empty() ? 0.2 : std::stod(arg2);
            log("CLIENT", "Loss rates set. Data (X) = " +
std::to_string(dataLossRate) +
                          ", ACK (Y) = " + std::to_string(ackLossRate));
            continue;
        }
```

由于通过文件传输验证实现的正确性，就取消了-testgbn，改为设置-setloss 函数，来统一设置 ACK 率和丢包率。

```
        // (2) 检查本地文件 (仅上传时)
        if (cmd == "upload" && !arg1.empty()) {
            std::ifstream testFile(arg1);
            if (!testFile) {
                log("CLIENT_ERROR", "Local file not found: " + arg1);
                continue;
            }
            testFile.close();
        }
        // (3) 发送命令到服务器
        sendto(clientSocket, input.c_str(), input.length(), 0,
(SOCKADDR*)&serverAddr, sizeof(serverAddr));
        if (cmd == "-quit") {
            log("CLIENT", "Quitting...");
        }
        // (4) 等待服务器回复
        sockaddr_in fromAddr;
        int fromAddrLen = sizeof(fromAddr);
        int recvLen = recvfrom(clientSocket, responseBuffer,
sizeof(responseBuffer) - 1, 0, (SOCKADDR*)&fromAddr, &fromAddrLen);
        if (recvLen == SOCKET_ERROR) {
            log("CLIENT_ERROR", "Server timed out.");
            continue;
        }

        responseBuffer[recvLen] = '\0';
        std::string response(responseBuffer);
        log("SERVER_REPLY", response);
        if (cmd == "-quit" && response == MSG_GOODBYE) {
            break; // 退出
        }
```

如果接收到 Good bye!回复就退出。

```
        // (5) 如果回复 OK，启动 GBN
```

```
        if (response == MSG_OK) {
            if (cmd == "upload" && !arg1.empty()) {
                // 启动 GBN 发送
                gbnSendFile(clientSocket, serverAddr, arg1,
"CLIENT_SEND");
            }
            else if (cmd == "download" && !arg1.empty()) {
                // 启动 GBN 接收 (带丢包模拟)
                std::string save_filename = "client_" + arg1;
                gbnReceiveFile(clientSocket, serverAddr, save_filename,
"CLIENT_RECV", dataLossRate, ackLossRate);
            }
        }
    }
    closesocket(clientSocket);
    WSACleanup();
    return 0;
```

server.cpp

```
#include "../common.h"
#include "../gbn_logic.h" // 包含 GBN 逻辑

// 辅助函数：解析命令
void parseCommand(const std::string& raw, std::string& cmd, std::string&
filename) {
    std::stringstream ss(raw);
    ss >> cmd;
    ss >> filename;
}
```

```
int main() {
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        log("SERVER_ERROR", "WSAStartup failed.");
        return 1;
    }
    SOCKET serverSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    setSocketTimeout(serverSocket, 0); // 默认阻塞等待命令
```

```
    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(SERVER_PORT);
    if (bind(serverSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
```

```
SOCKET_ERROR) {
        log("SERVER_ERROR", "Bind failed.");
        closesocket(serverSocket); WSACleanup(); return 1;
    }
    log("SERVER", "GBN Server running, waiting for commands on port " +
std::to_string(SERVER_PORT));
```

初始化网络设置。

```
    char cmdBuffer[1024];
    sockaddr_in clientAddr;
    int clientAddrLen = sizeof(clientAddr);

    while (true) {
        // 1. 阻塞等待命令
        log("SERVER", "Waiting for new command...");
        int recvLen = recvfrom(serverSocket, cmdBuffer, sizeof(cmdBuffer)
- 1, 0, (SOCKADDR*)&clientAddr, &clientAddrLen);
        if (recvLen <= 0) continue;
        cmdBuffer[recvLen] = '\0';

        std::string rawCmd(cmdBuffer);
        std::string cmd, filename;
        parseCommand(rawCmd, cmd, filename);
        log("SERVER", "Received command: '" + cmd + "' Filename: '" + filename
+ "'");
        std::string response;
        if (cmd == "-time") {
            response = "Server time: " + getCurrentTime();
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
        }
```

如果是“-time”指令就发送当前时间

```
        else if (cmd == "-quit") {
            response = MSG_GOODBYE;
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
        }
```

如果是“-quit”指令就发送 Good bye!

```
        else if (cmd == "upload" && !filename.empty()) {
            // 客户端请求上传
            std::string save_filename = "server_" + filename;
            response = MSG_OK;
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
```

```
            // 进入 GBN 接收逻辑 (服务器接收时不模拟丢包)
            gbnReceiveFile(serverSocket, clientAddr, save_filename,
"SERVER_RECV", 0.0, 0.0);
        }
        else if (cmd == "download" && !filename.empty()) {
            // 客户端请求下载
            std::ifstream testFile(filename);
            if (!testFile) {
                response = std::string(MSG_ERR) + ": File not found";
                sendto(serverSocket, response.c_str(), response.length(),
0, (SOCKADDR*)&clientAddr, clientAddrLen);
            } else {
                testFile.close();
                response = MSG_OK;
                sendto(serverSocket, response.c_str(), response.length(),
0, (SOCKADDR*)&clientAddr, clientAddrLen);

                // 进入 GBN 发送逻辑
                gbnSendFile(serverSocket, clientAddr, filename,
"SERVER_SEND");
            }
        }
```

否则为未定义命令，发送回去。

```
        else {
            response = std::string(MSG_ERR) + ": Unknown command";
            sendto(serverSocket, response.c_str(), response.length(), 0,
(SOCKADDR*)&clientAddr, clientAddrLen);
        }
    }
    closesocket(serverSocket);
    WSACleanup();
    return 0;
}
```

在改为使用 SR 协议后，对于 share.h，其内容和 common.h 没有区别。所以不多赘述。对于 client.cpp 和 server.cpp，main 函数的内容也没有明显区别，只是调用的接口不同，同样也不多言。重点看 SR 协议实现的 SR.h 这一头文件。

发送函数

```
bool srSendFile(SOCKET sock, const sockaddr_in& clientAddr, const
std::string& filename, const std::string& logPrefix)
```

重点是发送逻辑不同

```
    while (base < totalFrames) {
        // (1) [发送]
        while (nextSeqNum < totalFrames) {
```

```
            uint8_t distance = (nextSeqNum - base + 256) % 256;
            if (distance >= WINDOW_SIZE) break;

            const auto& frameToSend = allFrames[nextSeqNum];
            log(logPrefix, "SEND Data " + std::to_string(nextSeqNum));
            sendto(sock, frameToSend.data(), MAX_FRAME_SIZE, 0,
(SOCKADDR*)&clientAddr, sizeof(clientAddr));

            packet_timers[nextSeqNum] = std::chrono::steady_clock::now();
            nextSeqNum++;
            logSRSendWindow(base, nextSeqNum, totalFrames, acked_packets,
logPrefix); // 输出窗口变化
        }
```

由于 SR 协议是对每个数据帧进行独立超市检验，因此需要发送时需要将数据帧对应的发送时间节点记录下来，以便后续进行超市检验时进行计算。

```
        // (2) [接收 ACK]
        int recvLen = recvfrom(sock, ackBuffer, ACK_FRAME_SIZE, 0,
(SOCKADDR*)&fromAddr, &fromAddrLen);
        if (recvLen == ACK_FRAME_SIZE) {
            uint8_t ackNum = static_cast<uint8_t>(ackBuffer[0]);
            log(logPrefix, "RECV ACK " + std::to_string(ackNum));

            uint8_t distance = (ackNum - base + 256) % 256;
            if (distance < WINDOW_SIZE) { // 必须在窗口内
                if (acked_packets.find(ackNum) == acked_packets.end()) {
                     acked_packets.insert(ackNum);
                     packet_timers.erase(ackNum); // 停止计时
                     log(logPrefix, "Marked Data " + std::to_string(ackNum)
+ " as ACKed.");
                }
```

如果收到对应序列号的数据帧就停止计时并消除对应的数据帧的映射。

```
                if (ackNum == base) {
                    while (acked_packets.count(base) && base < totalFrames)
{
                        acked_packets.erase(base);
                        base++;
                    }
                    log(logPrefix, "SEND WINDOW base slides to " +
std::to_string(base));
                }
                logSRSendWindow(base, nextSeqNum, totalFrames,
acked_packets, logPrefix); // 输出窗口变化
            }
```

```
        } else { /* 1ms 轮询超时 */ }
```

base 数据包收到了 ACK，需要改变，考虑到 SR 协议要求缓存接收到的乱序帧，因此需要考虑连续更新的情况。

```
        // (3) [检查超时]
        auto now = std::chrono::steady_clock::now();
        for (auto it = packet_timers.begin(); it != packet_timers.end(); )
{
            uint8_t seq = it->first;
            auto duration =
std::chrono::duration_cast<std::chrono::milliseconds>(now -
it->second).count();
            if (duration > TIMEOUT_MS) {
                log(logPrefix, "TIMEOUT for Data " + std::to_string(seq) +
"! Resending.");
                const auto& frameToResend = allFrames[seq];
                sendto(sock, frameToResend.data(), MAX_FRAME_SIZE, 0,
(SOCKADDR*)&clientAddr, sizeof(clientAddr));
                packet_timers[seq] = now; // 重启定时器

                logSRSendWindow(base, nextSeqNum, totalFrames,
acked_packets, logPrefix); // 输出窗口 (重传)
                ++it;
            } else {
                ++it;
            }
        }
        Sleep(1);
    }
```

检查每个数据帧是否超时，出现超时就启动对该帧的重传。

接收函数

```
bool srReceiveFile(SOCKET sock, const sockaddr_in& serverAddr, const
std::string& filename,
                   const std::string& logPrefix, double dataLossRate,
double ackLossRate)
```

关键在于分类讨论接受情况。

```
        // (Case A: 按序或在缓冲区)
        if (seqNum == recv_base) {
            log(logPrefix, "DELIVER Data " + std::to_string(seqNum));
            // (模拟 ACK 丢失)
            if ((static_cast<double>(rand()) / RAND_MAX) < ackLossRate)
log(logPrefix, "SIMULATE ACK LOSS for " + std::to_string(seqNum));
            else srSendAck(sock, serverAddr, seqNum, logPrefix);
```

```
            // 交付 base
            if (!isEof) outFile.write(&recvBuffer[FRAME_HEADER_SIZE +
FRAME_CHECKSUM_SIZE], DATA_PAYLOAD_SIZE);
            else finReceived = true;

            recv_base++;
            // 检查缓冲区，交付所有连续的包
            while (recv_buffer.count(recv_base)) {
                std::vector<char>& bufferedFrame = recv_buffer[recv_base];
                log(logPrefix, "DELIVER (from buffer) Data " +
std::to_string(recv_base));
                bool bufferedEof = (bufferedFrame[MAX_FRAME_SIZE - 1] ==
FLAG_EOF);
                if (!bufferedEof)
outFile.write(&bufferedFrame[FRAME_HEADER_SIZE + FRAME_CHECKSUM_SIZE],
DATA_PAYLOAD_SIZE);
                else finReceived = true;

                recv_buffer.erase(recv_base);
                recv_base++;
            }
            logSRRecvWindow(recv_base, recv_buffer, logPrefix); // 输出窗
口变化
        }
        // (Case B: 乱序但在窗口内)
        else {
            uint8_t distance = (seqNum - recv_base + 256) % 256;
            if (distance < WINDOW_SIZE) {
                log(logPrefix, "BUFFER (Out of Order) Data " +
std::to_string(seqNum));
                if ((static_cast<double>(rand()) / RAND_MAX) < ackLossRate)
log(logPrefix, "SIMULATE ACK LOSS for " + std::to_string(seqNum));
                else srSendAck(sock, serverAddr, seqNum, logPrefix); // SR:
确认收到的包

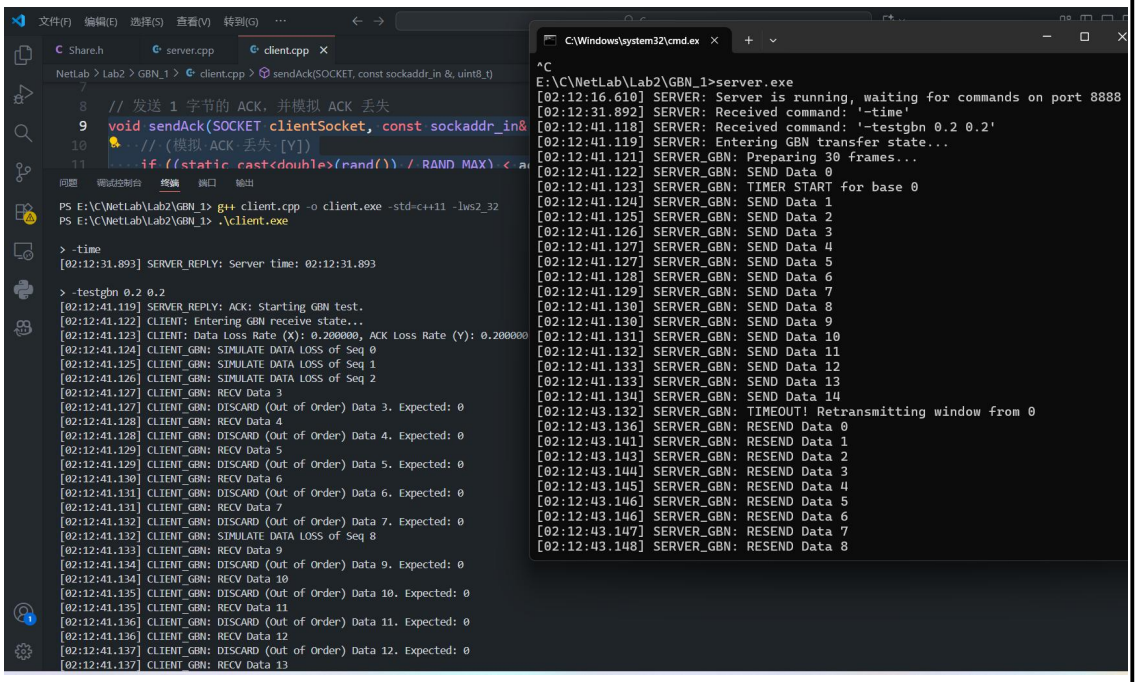
                if (recv_buffer.find(seqNum) == recv_buffer.end()) {
                    recv_buffer[seqNum] = std::vector<char>(recvBuffer,
recvBuffer + MAX_FRAME_SIZE);
                }
                logSRRecvWindow(recv_base, recv_buffer, logPrefix); // 输
出窗口变化（缓存）
            }
            // (Case C: 窗口外的旧包)
```

```
            else if (distance >= (256 - WINDOW_SIZE)) {
                log(logPrefix, "RECV (Old) Data " +
std::to_string(seqNum));
                if ((static_cast<double>(rand()) / RAND_MAX) <
ackLossRate) log(logPrefix, "SIMULATE ACK LOSS for (Old) " +
std::to_string(seqNum));
                else srSendAck(sock, serverAddr, seqNum, logPrefix);
            }
            else { /* 丢弃 */ }
        }

        if (finReceived && recv_buffer.empty()) { log(logPrefix, "EOF
received and buffer empty. Transfer complete."); break; }
    }
    outFile.close();
```

如果接收到的包按序，则将已缓存的连续的包一同交付上层。若为乱序，则缓存，如果不在窗口中就丢弃。

实验结果：



先进行第一部分的内容验收测试。

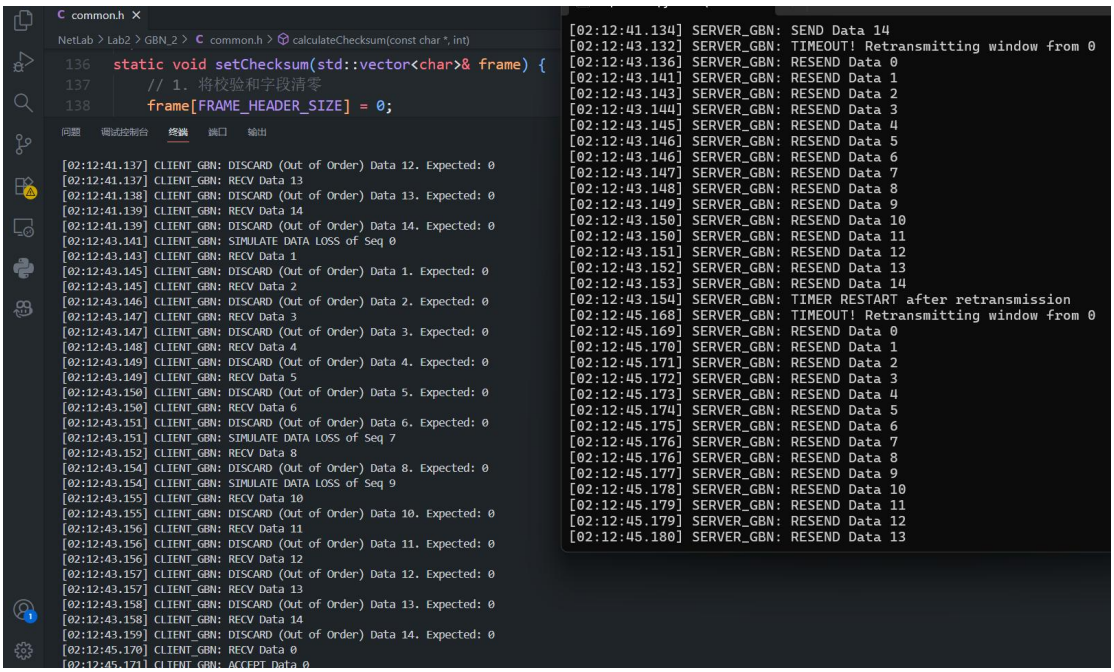
输入-time指令可以看到左侧成功打印出当前时间。

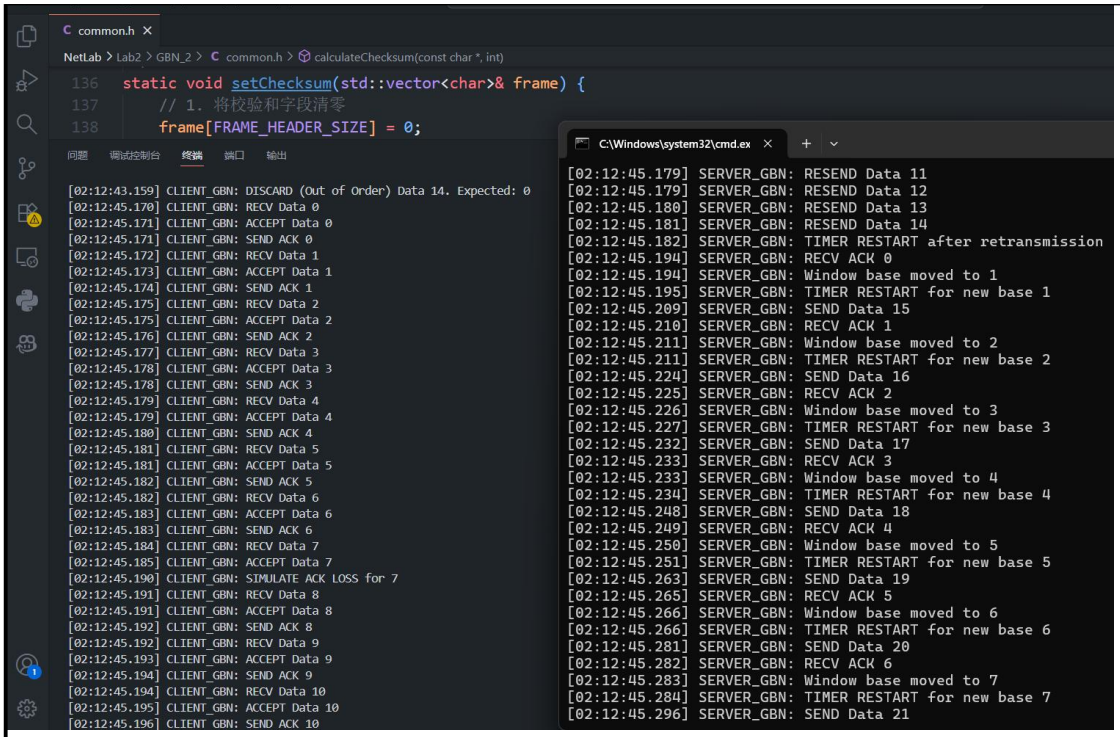
输入-testgbn 0.2 0.2指令，设置ACK、数据丢包率均为0.2。

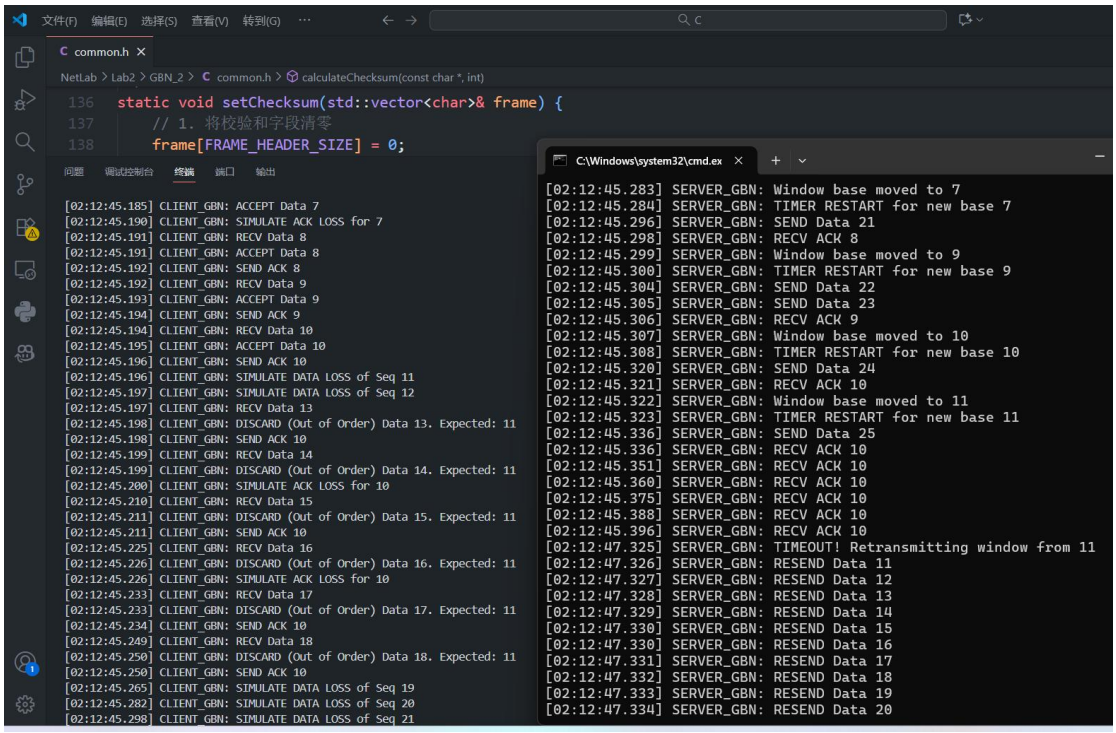
设置丢包率完毕后，开始模拟。

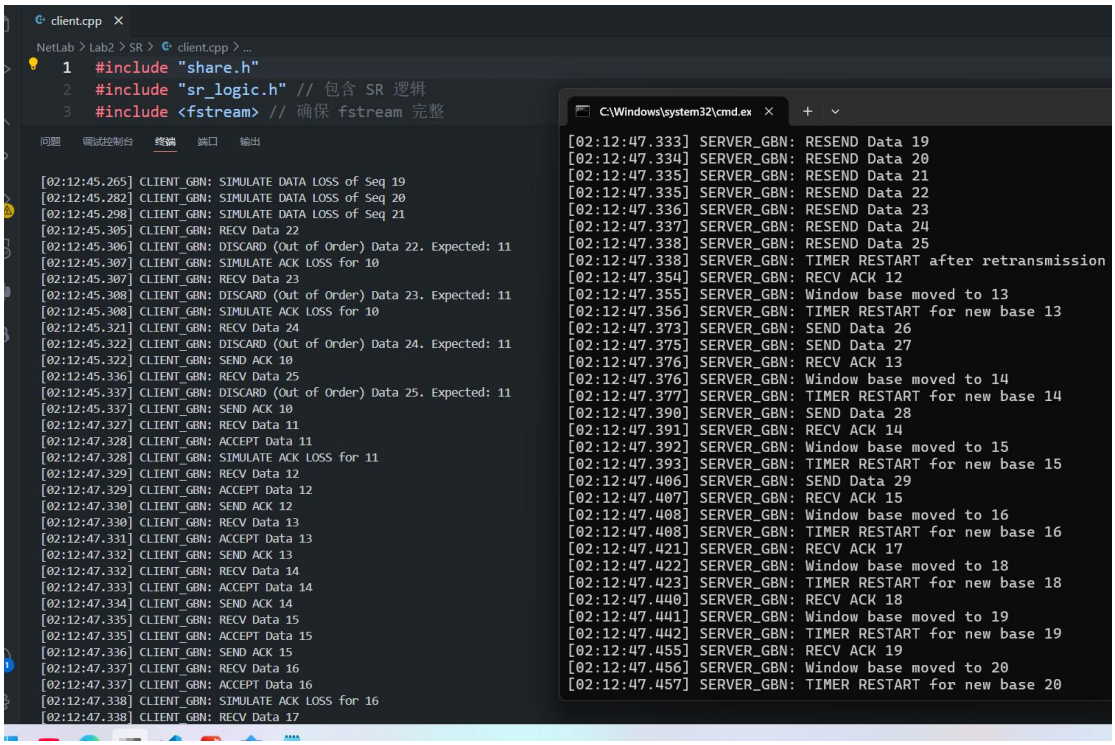
服务器端流水线发送了序号从0——14的数据帧。而服务器端模拟了数据帧0、1、2的丢失，因此在将要发送第15个数据帧时发生重传。在客户端等待0号数据帧过程中接收到了序列号为3到7、9到14的数据帧，但因为没有按序接收到，因此全部丢弃，直至超时触发重传。结

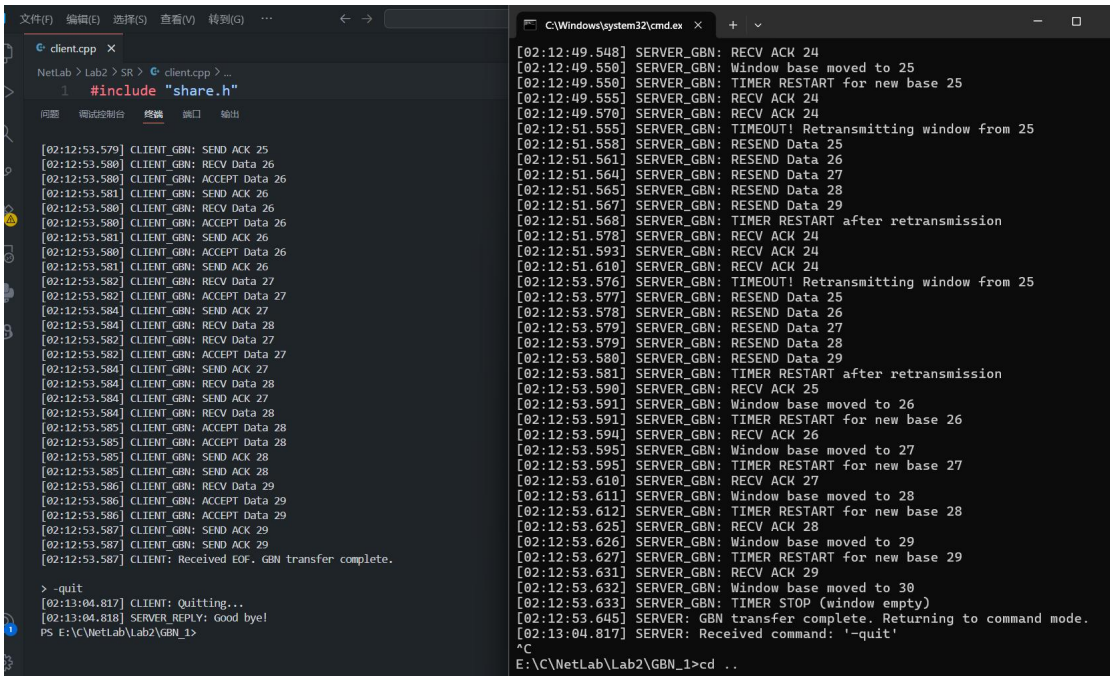
果在重传的开始发送0号数据帧时又触发了0号数据帧的丢失，因此服务器端又重新发送了窗口内序列号为0——14的数据帧。客户端在等待重传的数据帧到达的过程中收到了为按顺序到达的1——14号数据帧，均选择了忽略，最后客户端接收到了来自服务器重发的数据帧0，并在之后都顺序接收一直到7时，发生对7的ACK的丢失。这个过程中，在服务器端，由于窗口base不断移动，计时器一直重启，未触发重传，由于窗口移动使得服务器可发送的数据帧不断增加，当base移动到7时，已完成对21号数据帧的发送。虽然对7的ACK发送失败，但是对数据帧8的ACK的接收没有失败，所以，由于GBN采用累计确认，因此认为数据帧8及以前的数据帧都已成功接收，看到base移动到9，说明已发送但还未确认的数据帧中最早的是数据帧9。验证了累计确认机制。然后，继续接收数据帧，一直到11号数据帧时发生了11、12号数据帧的丢失，在等待11号数据帧的过程中，接收到了13号到8号数据帧，但由于失序到达，因此发生重传，同时也不断发送对10号数据帧的ACK，并且被服务器接收到。服务器则继续发送窗口内的数据帧，一直发送到25号数据帧。最终触发超时重传机制，服务器重新发送了从11号数据帧到到25号数据帧。在这次重传中，服务器端完成了从11号到20号数据帧的接收，但是16号的ACK丢失，不过可以看到，由于之后的17号数据帧的ACK成功被接收，所以根据累计确认，base正常移动到了18号的位置，并继续接收，一直移动到21号。21号数据发生丢失，所以直接开始等待，之后接收到的数据帧一律丢弃。在此之前，服务器已经发送全部数据帧开始空转，并在接收到20号ACK后开始等待直至超时。超时后，重发了从21号开始到29号的全部数据帧。在发送到25号数据帧后重传，结果又遇到了第二次25号数据帧丢失，发生又一次超时，重传了25号开始到29号截止的数据帧，最终完成了全部传输。最后服务器发送了EOF0帧，告诉客户端数据传输完毕，客户端结束数据传输状态。在这之后，客户端输入-quit指令，可以看到服务器端接收到了-quit指令，并向客户端发送了Good bye!，客户端打印信息后，结束程序，自动退出。



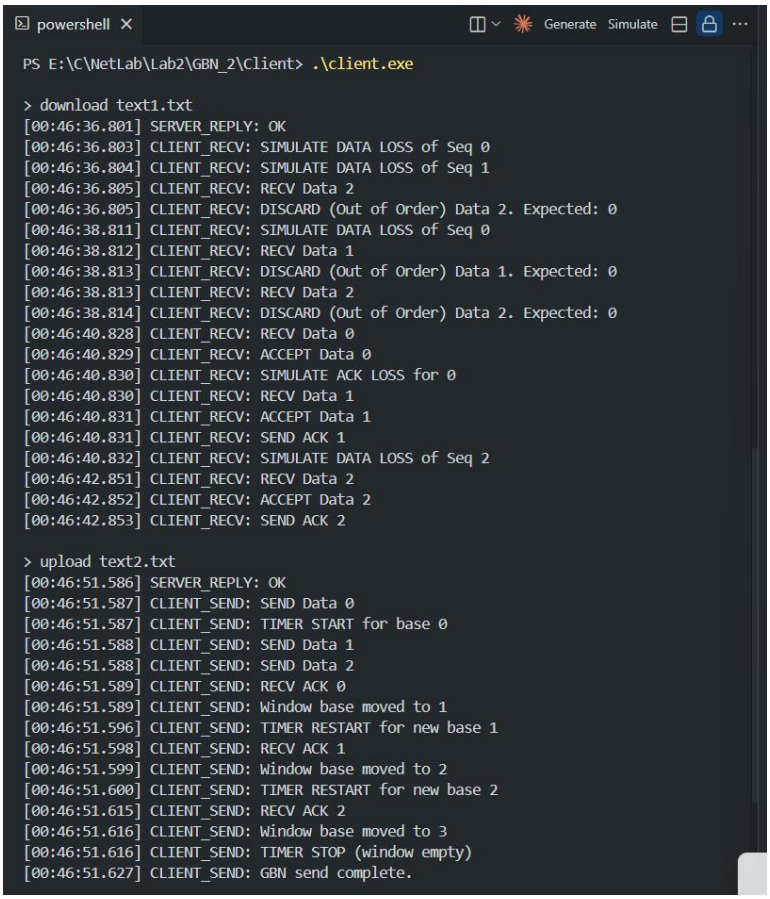

common.h
NetLab > Lab2 > GBN_2 > common.h > calculateChecksum(const char *, int)
136 static void setChecksum(std::vector<char>& frame) {
137 // 1. 将校验和字段清零
138 frame[FRAME_HEADER_SIZE] = 0;
问题 调试控制台 终端 端口 输出
[02:12:43.159] CLIENT_GBN: DISCARD (Out of Order) Data 14. Expected: 0
[02:12:45.170] CLIENT_GBN: RECV Data 0
[02:12:45.171] CLIENT_GBN: ACCEPT Data 0
[02:12:45.171] CLIENT_GBN: SEND ACK 0
[02:12:45.172] CLIENT_GBN: RECV Data 1
[02:12:45.173] CLIENT_GBN: ACCEPT Data 1
[02:12:45.174] CLIENT_GBN: SEND ACK 1
[02:12:45.175] CLIENT_GBN: RECV Data 2
[02:12:45.175] CLIENT_GBN: ACCEPT Data 2
[02:12:45.176] CLIENT_GBN: SEND ACK 2
[02:12:45.177] CLIENT_GBN: RECV Data 3
[02:12:45.178] CLIENT_GBN: ACCEPT Data 3
[02:12:45.178] CLIENT_GBN: SEND ACK 3
[02:12:45.179] CLIENT_GBN: RECV Data 4
[02:12:45.179] CLIENT_GBN: ACCEPT Data 4
[02:12:45.180] CLIENT_GBN: SEND ACK 4
[02:12:45.181] CLIENT_GBN: RECV Data 5
[02:12:45.181] CLIENT_GBN: ACCEPT Data 5
[02:12:45.182] CLIENT_GBN: SEND ACK 5
[02:12:45.182] CLIENT_GBN: RECV Data 6
[02:12:45.183] CLIENT_GBN: ACCEPT Data 6
[02:12:45.183] CLIENT_GBN: SEND ACK 6
[02:12:45.184] CLIENT_GBN: RECV Data 7
[02:12:45.185] CLIENT_GBN: ACCEPT Data 7
[02:12:45.190] CLIENT_GBN: SIMULATE ACK LOSS for 7
[02:12:45.191] CLIENT_GBN: RECV Data 8
[02:12:45.191] CLIENT_GBN: ACCEPT Data 8
[02:12:45.192] CLIENT_GBN: SEND ACK 8
[02:12:45.192] CLIENT_GBN: RECV Data 9
[02:12:45.193] CLIENT_GBN: ACCEPT Data 9
[02:12:45.194] CLIENT_GBN: SEND ACK 9
[02:12:45.194] CLIENT_GBN: RECV Data 10
[02:12:45.195] CLIENT_GBN: ACCEPT Data 10
[02:12:45.196] CLIENT_GBN: SEND ACK 10
C:\Windows\system32\cmd.ex
[02:12:45.179] SERVER_GBN: RESEND Data 11
[02:12:45.179] SERVER_GBN: RESEND Data 12
[02:12:45.180] SERVER_GBN: RESEND Data 13
[02:12:45.181] SERVER_GBN: RESEND Data 14
[02:12:45.182] SERVER_GBN: TIMER RESTART after retransmission
[02:12:45.194] SERVER_GBN: RECV ACK 0
[02:12:45.194] SERVER_GBN: Window base moved to 1
[02:12:45.195] SERVER_GBN: TIMER RESTART for new base 1
[02:12:45.209] SERVER_GBN: SEND Data 15
[02:12:45.210] SERVER_GBN: RECV ACK 1
[02:12:45.211] SERVER_GBN: Window base moved to 2
[02:12:45.211] SERVER_GBN: TIMER RESTART for new base 2
[02:12:45.224] SERVER_GBN: SEND Data 16
[02:12:45.225] SERVER_GBN: RECV ACK 2
[02:12:45.226] SERVER_GBN: Window base moved to 3
[02:12:45.227] SERVER_GBN: TIMER RESTART for new base 3
[02:12:45.232] SERVER_GBN: SEND Data 17
[02:12:45.233] SERVER_GBN: RECV ACK 3
[02:12:45.233] SERVER_GBN: Window base moved to 4
[02:12:45.234] SERVER_GBN: TIMER RESTART for new base 4
[02:12:45.248] SERVER_GBN: SEND Data 18
[02:12:45.249] SERVER_GBN: RECV ACK 4
[02:12:45.250] SERVER_GBN: Window base moved to 5
[02:12:45.251] SERVER_GBN: TIMER RESTART for new base 5
[02:12:45.263] SERVER_GBN: SEND Data 19
[02:12:45.265] SERVER_GBN: RECV ACK 5
[02:12:45.266] SERVER_GBN: Window base moved to 6
[02:12:45.266] SERVER_GBN: TIMER RESTART for new base 6
[02:12:45.281] SERVER_GBN: SEND Data 20
[02:12:45.282] SERVER_GBN: RECV ACK 6
[02:12:45.283] SERVER_GBN: Window base moved to 7
[02:12:45.284] SERVER_GBN: TIMER RESTART for new base 7
[02:12:45.296] SERVER_GBN: SEND Data 21

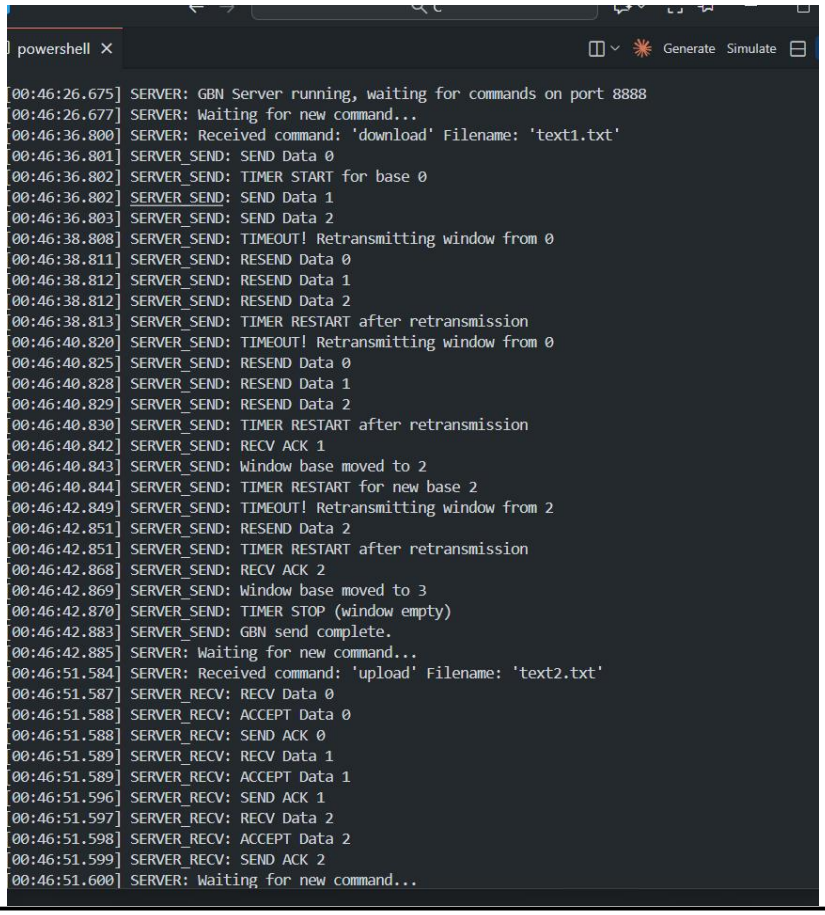

文件(F) 编辑(E) 选择(S) 查看(V) 转到(G)
common.h
NetLab > Lab2 > GBN_2 > common.h > calculateChecksum(const char *, int)
136 static void setChecksum(std::vector<char>& frame) {
137 // 1. 将校验和字段清零
138 frame[FRAME_HEADER_SIZE] = 0;
问题 调试控制台 终端 端口 输出
[02:12:45.185] CLIENT_GBN: ACCEPT Data 7
[02:12:45.190] CLIENT_GBN: SIMULATE ACK LOSS for 7
[02:12:45.191] CLIENT_GBN: RECV Data 8
[02:12:45.191] CLIENT_GBN: ACCEPT Data 8
[02:12:45.192] CLIENT_GBN: SEND ACK 8
[02:12:45.192] CLIENT_GBN: RECV Data 9
[02:12:45.193] CLIENT_GBN: ACCEPT Data 9
[02:12:45.194] CLIENT_GBN: SEND ACK 9
[02:12:45.194] CLIENT_GBN: RECV Data 10
[02:12:45.195] CLIENT_GBN: ACCEPT Data 10
[02:12:45.196] CLIENT_GBN: SEND ACK 10
[02:12:45.196] CLIENT_GBN: SIMULATE DATA LOSS of Seq 11
[02:12:45.197] CLIENT_GBN: SIMULATE DATA LOSS of Seq 12
[02:12:45.197] CLIENT_GBN: RECV Data 13
[02:12:45.198] CLIENT_GBN: DISCARD (Out of Order) Data 13. Expected: 11
[02:12:45.198] CLIENT_GBN: SEND ACK 10
[02:12:45.199] CLIENT_GBN: RECV Data 14
[02:12:45.199] CLIENT_GBN: DISCARD (Out of Order) Data 14. Expected: 11
[02:12:45.200] CLIENT_GBN: SIMULATE ACK LOSS for 10
[02:12:45.210] CLIENT_GBN: RECV Data 15
[02:12:45.211] CLIENT_GBN: DISCARD (Out of Order) Data 15. Expected: 11
[02:12:45.211] CLIENT_GBN: SEND ACK 10
[02:12:45.225] CLIENT_GBN: RECV Data 16
[02:12:45.226] CLIENT_GBN: DISCARD (Out of Order) Data 16. Expected: 11
[02:12:45.226] CLIENT_GBN: SIMULATE ACK LOSS for 10
[02:12:45.233] CLIENT_GBN: RECV Data 17
[02:12:45.233] CLIENT_GBN: DISCARD (Out of Order) Data 17. Expected: 11
[02:12:45.234] CLIENT_GBN: SEND ACK 10
[02:12:45.249] CLIENT_GBN: RECV Data 18
[02:12:45.250] CLIENT_GBN: DISCARD (Out of Order) Data 18. Expected: 11
[02:12:45.250] CLIENT_GBN: SEND ACK 10
[02:12:45.265] CLIENT_GBN: SIMULATE DATA LOSS of Seq 19
[02:12:45.282] CLIENT_GBN: SIMULATE DATA LOSS of Seq 20
[02:12:45.298] CLIENT_GBN: SIMULATE DATA LOSS of Seq 21
C:\Windows\system32\cmd.ex
[02:12:45.283] SERVER_GBN: Window base moved to 7
[02:12:45.284] SERVER_GBN: TIMER RESTART for new base 7
[02:12:45.296] SERVER_GBN: SEND Data 21
[02:12:45.298] SERVER_GBN: RECV ACK 8
[02:12:45.299] SERVER_GBN: Window base moved to 9
[02:12:45.300] SERVER_GBN: TIMER RESTART for new base 9
[02:12:45.304] SERVER_GBN: SEND Data 22
[02:12:45.305] SERVER_GBN: SEND Data 23
[02:12:45.306] SERVER_GBN: RECV ACK 9
[02:12:45.307] SERVER_GBN: Window base moved to 10
[02:12:45.308] SERVER_GBN: TIMER RESTART for new base 10
[02:12:45.320] SERVER_GBN: SEND Data 24
[02:12:45.321] SERVER_GBN: RECV ACK 10
[02:12:45.322] SERVER_GBN: Window base moved to 11
[02:12:45.323] SERVER_GBN: TIMER RESTART for new base 11
[02:12:45.336] SERVER_GBN: SEND Data 25
[02:12:45.336] SERVER_GBN: RECV ACK 10
[02:12:45.351] SERVER_GBN: RECV ACK 10
[02:12:45.360] SERVER_GBN: RECV ACK 10
[02:12:45.375] SERVER_GBN: RECV ACK 10
[02:12:45.388] SERVER_GBN: RECV ACK 10
[02:12:45.396] SERVER_GBN: RECV ACK 10
[02:12:47.325] SERVER_GBN: TIMEOUT! Retransmitting window from 11
[02:12:47.326] SERVER_GBN: RESEND Data 11
[02:12:47.327] SERVER_GBN: RESEND Data 12
[02:12:47.328] SERVER_GBN: RESEND Data 13
[02:12:47.329] SERVER_GBN: RESEND Data 14
[02:12:47.330] SERVER_GBN: RESEND Data 15
[02:12:47.330] SERVER_GBN: RESEND Data 16
[02:12:47.331] SERVER_GBN: RESEND Data 17
[02:12:47.332] SERVER_GBN: RESEND Data 18
[02:12:47.333] SERVER_GBN: RESEND Data 19
[02:12:47.334] SERVER_GBN: RESEND Data 20


client.cpp
NetLab > Lab2 > SR > client.cpp > ...
1 #include "share.h"
2 #include "sr_logic.h" // 包含 SR 逻辑
3 #include <fstream> // 确保 fstream 完整
问题 调试控制台 终端 端口 输出
[02:12:45.265] CLIENT_GBN: SIMULATE DATA LOSS of Seq 19
[02:12:45.282] CLIENT_GBN: SIMULATE DATA LOSS of Seq 20
[02:12:45.298] CLIENT_GBN: SIMULATE DATA LOSS of Seq 21
[02:12:45.305] CLIENT_GBN: RECV Data 22
[02:12:45.306] CLIENT_GBN: DISCARD (Out of Order) Data 22. Expected: 11
[02:12:45.307] CLIENT_GBN: SIMULATE ACK LOSS for 10
[02:12:45.307] CLIENT_GBN: RECV Data 23
[02:12:45.308] CLIENT_GBN: DISCARD (Out of Order) Data 23. Expected: 11
[02:12:45.308] CLIENT_GBN: SIMULATE ACK LOSS for 10
[02:12:45.321] CLIENT_GBN: RECV Data 24
[02:12:45.322] CLIENT_GBN: DISCARD (Out of Order) Data 24. Expected: 11
[02:12:45.322] CLIENT_GBN: SEND ACK 10
[02:12:45.336] CLIENT_GBN: RECV Data 25
[02:12:45.337] CLIENT_GBN: DISCARD (Out of Order) Data 25. Expected: 11
[02:12:45.337] CLIENT_GBN: SEND ACK 10
[02:12:47.327] CLIENT_GBN: RECV Data 11
[02:12:47.328] CLIENT_GBN: ACCEPT Data 11
[02:12:47.328] CLIENT_GBN: SIMULATE ACK LOSS for 11
[02:12:47.329] CLIENT_GBN: RECV Data 12
[02:12:47.329] CLIENT_GBN: ACCEPT Data 12
[02:12:47.330] CLIENT_GBN: SEND ACK 12
[02:12:47.330] CLIENT_GBN: RECV Data 13
[02:12:47.331] CLIENT_GBN: ACCEPT Data 13
[02:12:47.332] CLIENT_GBN: SEND ACK 13
[02:12:47.332] CLIENT_GBN: RECV Data 14
[02:12:47.333] CLIENT_GBN: ACCEPT Data 14
[02:12:47.334] CLIENT_GBN: SEND ACK 14
[02:12:47.335] CLIENT_GBN: RECV Data 15
[02:12:47.335] CLIENT_GBN: ACCEPT Data 15
[02:12:47.336] CLIENT_GBN: SEND ACK 15
[02:12:47.337] CLIENT_GBN: RECV Data 16
[02:12:47.337] CLIENT_GBN: ACCEPT Data 16
[02:12:47.338] CLIENT_GBN: SIMULATE ACK LOSS for 16
[02:12:47.338] CLIENT_GBN: RECV Data 17
C:\Windows\system32\cmd.ex
[02:12:47.333] SERVER_GBN: RESEND Data 19
[02:12:47.334] SERVER_GBN: RESEND Data 20
[02:12:47.335] SERVER_GBN: RESEND Data 21
[02:12:47.335] SERVER_GBN: RESEND Data 22
[02:12:47.336] SERVER_GBN: RESEND Data 23
[02:12:47.337] SERVER_GBN: RESEND Data 24
[02:12:47.338] SERVER_GBN: RESEND Data 25
[02:12:47.338] SERVER_GBN: TIMER RESTART after retransmission
[02:12:47.354] SERVER_GBN: RECV ACK 12
[02:12:47.355] SERVER_GBN: Window base moved to 13
[02:12:47.356] SERVER_GBN: TIMER RESTART for new base 13
[02:12:47.373] SERVER_GBN: SEND Data 26
[02:12:47.375] SERVER_GBN: SEND Data 27
[02:12:47.376] SERVER_GBN: RECV ACK 13
[02:12:47.376] SERVER_GBN: Window base moved to 14
[02:12:47.377] SERVER_GBN: TIMER RESTART for new base 14
[02:12:47.390] SERVER_GBN: SEND Data 28
[02:12:47.391] SERVER_GBN: RECV ACK 14
[02:12:47.392] SERVER_GBN: Window base moved to 15
[02:12:47.393] SERVER_GBN: TIMER RESTART for new base 15
[02:12:47.406] SERVER_GBN: SEND Data 29
[02:12:47.407] SERVER_GBN: RECV ACK 15
[02:12:47.408] SERVER_GBN: Window base moved to 16
[02:12:47.408] SERVER_GBN: TIMER RESTART for new base 16
[02:12:47.421] SERVER_GBN: RECV ACK 17
[02:12:47.422] SERVER_GBN: Window base moved to 18
[02:12:47.423] SERVER_GBN: TIMER RESTART for new base 18
[02:12:47.440] SERVER_GBN: RECV ACK 18
[02:12:47.441] SERVER_GBN: Window base moved to 19
[02:12:47.442] SERVER_GBN: TIMER RESTART for new base 19
[02:12:47.455] SERVER_GBN: RECV ACK 19
[02:12:47.456] SERVER_GBN: Window base moved to 20
[02:12:47.457] SERVER_GBN: TIMER RESTART for new base 20

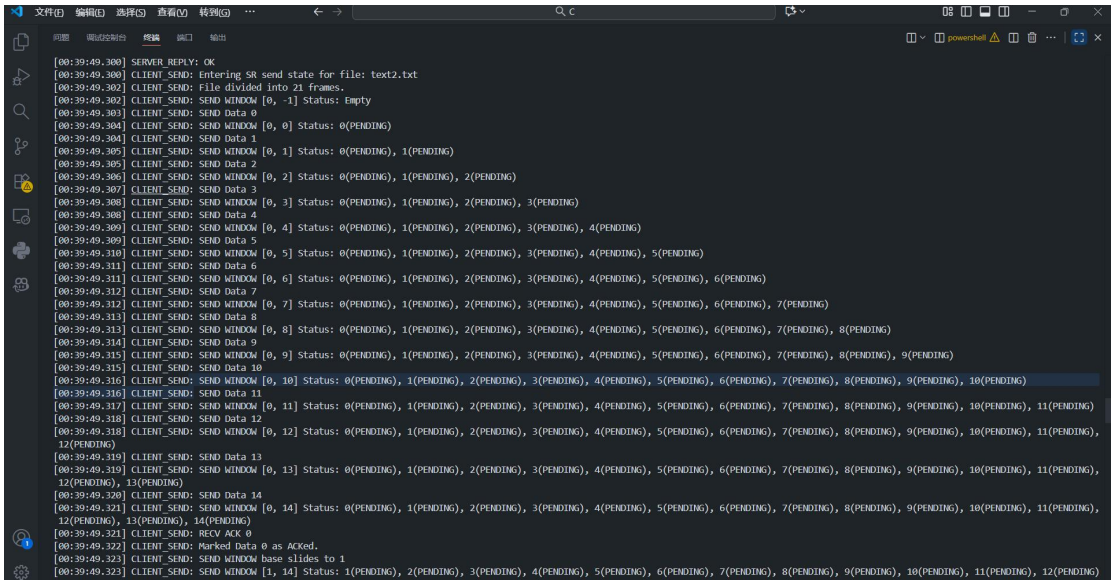

文件(F) 编辑(E) 选择(S) 查看(V) 转到(G)
client.cpp
NetLab > Lab2 > SR > client.cpp > ...
1 #include "share.h"
问题 调试控制台 终端 端口 输出
[02:12:53.579] CLIENT_GBN: SEND ACK 25
[02:12:53.580] CLIENT_GBN: RECV Data 26
[02:12:53.580] CLIENT_GBN: ACCEPT Data 26
[02:12:53.581] CLIENT_GBN: SEND ACK 26
[02:12:53.580] CLIENT_GBN: RECV Data 26
[02:12:53.580] CLIENT_GBN: ACCEPT Data 26
[02:12:53.581] CLIENT_GBN: SEND ACK 26
[02:12:53.580] CLIENT_GBN: ACCEPT Data 26
[02:12:53.581] CLIENT_GBN: SEND ACK 26
[02:12:53.582] CLIENT_GBN: RECV Data 27
[02:12:53.582] CLIENT_GBN: ACCEPT Data 27
[02:12:53.584] CLIENT_GBN: SEND ACK 27
[02:12:53.584] CLIENT_GBN: RECV Data 28
[02:12:53.582] CLIENT_GBN: RECV Data 27
[02:12:53.582] CLIENT_GBN: ACCEPT Data 27
[02:12:53.584] CLIENT_GBN: SEND ACK 27
[02:12:53.584] CLIENT_GBN: RECV Data 28
[02:12:53.584] CLIENT_GBN: SEND ACK 27
[02:12:53.584] CLIENT_GBN: RECV Data 28
[02:12:53.585] CLIENT_GBN: ACCEPT Data 28
[02:12:53.585] CLIENT_GBN: ACCEPT Data 28
[02:12:53.585] CLIENT_GBN: SEND ACK 28
[02:12:53.585] CLIENT_GBN: SEND ACK 28
[02:12:53.586] CLIENT_GBN: RECV Data 29
[02:12:53.586] CLIENT_GBN: ACCEPT Data 29
[02:12:53.586] CLIENT_GBN: ACCEPT Data 29
[02:12:53.587] CLIENT_GBN: SEND ACK 29
[02:12:53.587] CLIENT_GBN: SEND ACK 29
[02:12:53.587] CLIENT: Received EOF. GBN transfer complete.
> -quit
[02:13:04.817] CLIENT: Quitting...
[02:13:04.818] SERVER_REPLY: Good bye!
PS E:\C\NetLab\Lab2\GBN_1>
C:\Windows\system32\cmd.ex
[02:12:49.548] SERVER_GBN: RECV ACK 24
[02:12:49.550] SERVER_GBN: Window base moved to 25
[02:12:49.550] SERVER_GBN: TIMER RESTART for new base 25
[02:12:49.555] SERVER_GBN: RECV ACK 24
[02:12:49.570] SERVER_GBN: RECV ACK 24
[02:12:51.555] SERVER_GBN: TIMEOUT! Retransmitting window from 25
[02:12:51.558] SERVER_GBN: RESEND Data 25
[02:12:51.561] SERVER_GBN: RESEND Data 26
[02:12:51.564] SERVER_GBN: RESEND Data 27
[02:12:51.565] SERVER_GBN: RESEND Data 28
[02:12:51.567] SERVER_GBN: RESEND Data 29
[02:12:51.568] SERVER_GBN: TIMER RESTART after retransmission
[02:12:51.578] SERVER_GBN: RECV ACK 24
[02:12:51.593] SERVER_GBN: RECV ACK 24
[02:12:51.610] SERVER_GBN: RECV ACK 24
[02:12:53.576] SERVER_GBN: TIMEOUT! Retransmitting window from 25
[02:12:53.577] SERVER_GBN: RESEND Data 25
[02:12:53.578] SERVER_GBN: RESEND Data 26
[02:12:53.579] SERVER_GBN: RESEND Data 27
[02:12:53.579] SERVER_GBN: RESEND Data 28
[02:12:53.580] SERVER_GBN: RESEND Data 29
[02:12:53.581] SERVER_GBN: TIMER RESTART after retransmission
[02:12:53.590] SERVER_GBN: RECV ACK 25
[02:12:53.591] SERVER_GBN: Window base moved to 26
[02:12:53.591] SERVER_GBN: TIMER RESTART for new base 26
[02:12:53.594] SERVER_GBN: RECV ACK 26
[02:12:53.595] SERVER_GBN: Window base moved to 27
[02:12:53.595] SERVER_GBN: TIMER RESTART for new base 27
[02:12:53.610] SERVER_GBN: RECV ACK 27
[02:12:53.611] SERVER_GBN: Window base moved to 28
[02:12:53.612] SERVER_GBN: TIMER RESTART for new base 28
[02:12:53.625] SERVER_GBN: RECV ACK 28
[02:12:53.626] SERVER_GBN: Window base moved to 29
[02:12:53.627] SERVER_GBN: TIMER RESTART for new base 29
[02:12:53.631] SERVER_GBN: RECV ACK 29
[02:12:53.632] SERVER_GBN: Window base moved to 30
[02:12:53.633] SERVER_GBN: TIMER STOP (window empty)
[02:12:53.645] SERVER: GBN transfer complete. Returning to command mode.
[02:13:04.817] SERVER: Received command: '-quit'
^C
E:\C\NetLab\Lab2\GBN_1>cd ..

GBN应用测试

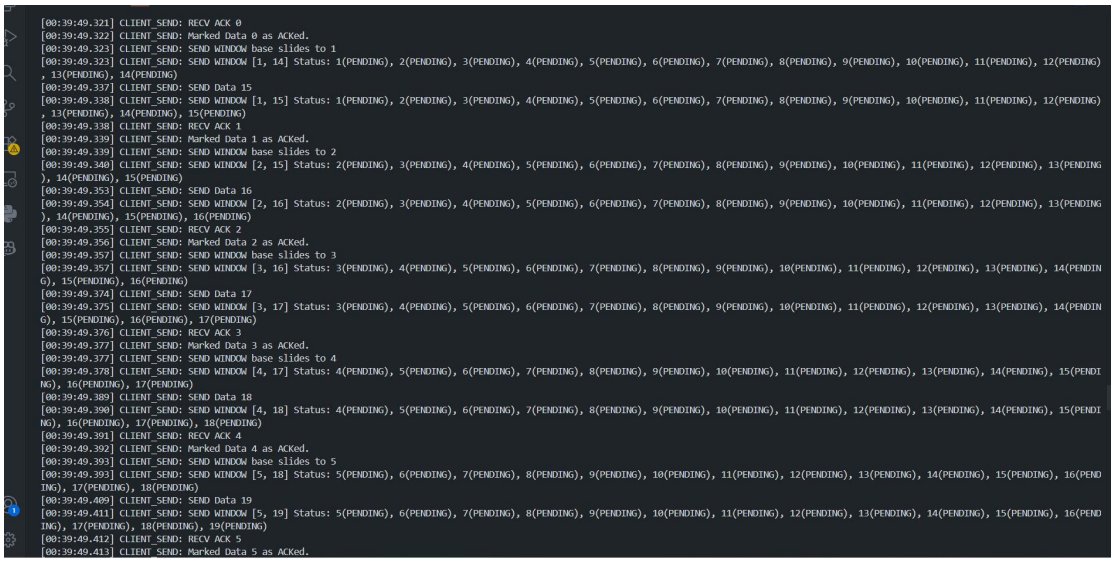

powershell
Generate
Simulate
PS E:\C\NetLab\Lab2\GBN_2\Client> .\client.exe
> download text1.txt
[00:46:36.801] SERVER_REPLY: OK
[00:46:36.803] CLIENT_RECV: SIMULATE DATA LOSS of Seq 0
[00:46:36.804] CLIENT_RECV: SIMULATE DATA LOSS of Seq 1
[00:46:36.805] CLIENT_RECV: RECV Data 2
[00:46:36.805] CLIENT_RECV: DISCARD (Out of Order) Data 2. Expected: 0
[00:46:38.811] CLIENT_RECV: SIMULATE DATA LOSS of Seq 0
[00:46:38.812] CLIENT_RECV: RECV Data 1
[00:46:38.813] CLIENT_RECV: DISCARD (Out of Order) Data 1. Expected: 0
[00:46:38.813] CLIENT_RECV: RECV Data 2
[00:46:38.814] CLIENT_RECV: DISCARD (Out of Order) Data 2. Expected: 0
[00:46:40.828] CLIENT_RECV: RECV Data 0
[00:46:40.829] CLIENT_RECV: ACCEPT Data 0
[00:46:40.830] CLIENT_RECV: SIMULATE ACK LOSS for 0
[00:46:40.830] CLIENT_RECV: RECV Data 1
[00:46:40.831] CLIENT_RECV: ACCEPT Data 1
[00:46:40.831] CLIENT_RECV: SEND ACK 1
[00:46:40.832] CLIENT_RECV: SIMULATE DATA LOSS of Seq 2
[00:46:42.851] CLIENT_RECV: RECV Data 2
[00:46:42.852] CLIENT_RECV: ACCEPT Data 2
[00:46:42.853] CLIENT_RECV: SEND ACK 2
> upload text2.txt
[00:46:51.586] SERVER_REPLY: OK
[00:46:51.587] CLIENT_SEND: SEND Data 0
[00:46:51.587] CLIENT_SEND: TIMER START for base 0
[00:46:51.588] CLIENT_SEND: SEND Data 1
[00:46:51.588] CLIENT_SEND: SEND Data 2
[00:46:51.589] CLIENT_SEND: RECV ACK 0
[00:46:51.589] CLIENT_SEND: Window base moved to 1
[00:46:51.596] CLIENT_SEND: TIMER RESTART for new base 1
[00:46:51.598] CLIENT_SEND: RECV ACK 1
[00:46:51.599] CLIENT_SEND: Window base moved to 2
[00:46:51.600] CLIENT_SEND: TIMER RESTART for new base 2
[00:46:51.615] CLIENT_SEND: RECV ACK 2
[00:46:51.616] CLIENT_SEND: Window base moved to 3
[00:46:51.616] CLIENT_SEND: TIMER STOP (window empty)
[00:46:51.627] CLIENT_SEND: GBN send complete.


powershell
Generate
Simulate
00:46:26.675] SERVER: GBN Server running, waiting for commands on port 8888
00:46:26.677] SERVER: Waiting for new command...
00:46:36.800] SERVER: Received command: 'download' Filename: 'text1.txt'
00:46:36.801] SERVER_SEND: SEND Data 0
00:46:36.802] SERVER_SEND: TIMER START for base 0
00:46:36.802] SERVER_SEND: SEND Data 1
00:46:36.803] SERVER_SEND: SEND Data 2
00:46:38.808] SERVER_SEND: TIMEOUT! Retransmitting window from 0
00:46:38.811] SERVER_SEND: RESEND Data 0
00:46:38.812] SERVER_SEND: RESEND Data 1
00:46:38.812] SERVER_SEND: RESEND Data 2
00:46:38.813] SERVER_SEND: TIMER RESTART after retransmission
00:46:40.820] SERVER_SEND: TIMEOUT! Retransmitting window from 0
00:46:40.825] SERVER_SEND: RESEND Data 0
00:46:40.828] SERVER_SEND: RESEND Data 1
00:46:40.829] SERVER_SEND: RESEND Data 2
00:46:40.830] SERVER_SEND: TIMER RESTART after retransmission
00:46:40.842] SERVER_SEND: RECV ACK 1
00:46:40.843] SERVER_SEND: Window base moved to 2
00:46:40.844] SERVER_SEND: TIMER RESTART for new base 2
00:46:42.849] SERVER_SEND: TIMEOUT! Retransmitting window from 2
00:46:42.851] SERVER_SEND: RESEND Data 2
00:46:42.851] SERVER_SEND: TIMER RESTART after retransmission
00:46:42.868] SERVER_SEND: RECV ACK 2
00:46:42.869] SERVER_SEND: Window base moved to 3
00:46:42.870] SERVER_SEND: TIMER STOP (window empty)
00:46:42.883] SERVER_SEND: GBN send complete.
00:46:42.885] SERVER: Waiting for new command...
00:46:51.584] SERVER: Received command: 'upload' Filename: 'text2.txt'
00:46:51.587] SERVER_RECV: RECV Data 0
00:46:51.588] SERVER_RECV: ACCEPT Data 0
00:46:51.588] SERVER_RECV: SEND ACK 0
00:46:51.589] SERVER_RECV: RECV Data 1
00:46:51.589] SERVER_RECV: ACCEPT Data 1
00:46:51.596] SERVER_RECV: SEND ACK 1
00:46:51.597] SERVER_RECV: RECV Data 2
00:46:51.598] SERVER_RECV: ACCEPT Data 2
00:46:51.599] SERVER_RECV: SEND ACK 2
00:46:51.600] SERVER: Waiting for new command...

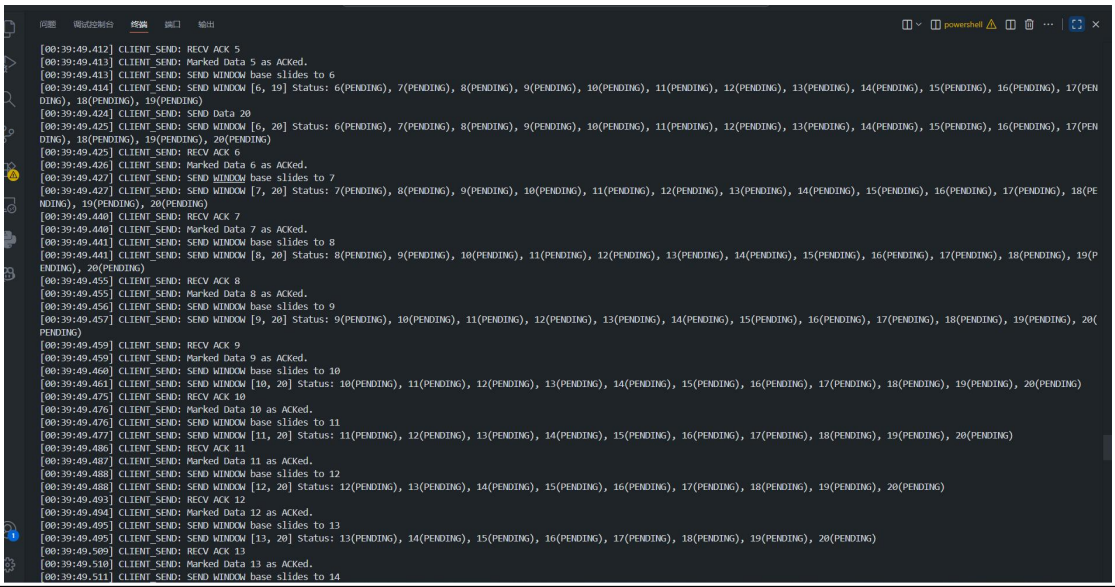
SR协议客户端打印信息



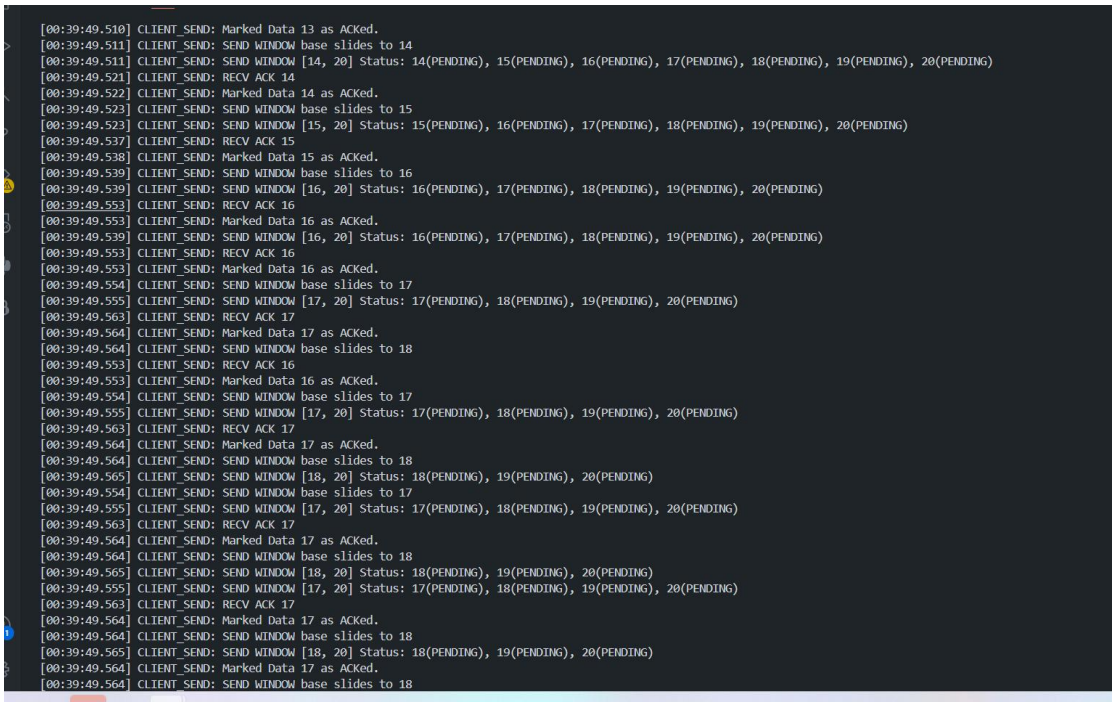
```
[00:39:49.300] SERVER_REPLY: OK
[00:39:49.300] CLIENT_SEND: Entering SR send state for file: text2.txt
[00:39:49.302] CLIENT_SEND: File divided into 21 frames.
[00:39:49.302] CLIENT_SEND: SEND WINDOW [0, -1] Status: Empty
[00:39:49.303] CLIENT_SEND: SEND Data 0
[00:39:49.304] CLIENT_SEND: SEND WINDOW [0, 0] Status: 0(PENDING)
[00:39:49.304] CLIENT_SEND: SEND Data 1
[00:39:49.305] CLIENT_SEND: SEND WINDOW [0, 1] Status: 0(PENDING), 1(PENDING)
[00:39:49.305] CLIENT_SEND: SEND Data 2
[00:39:49.306] CLIENT_SEND: SEND WINDOW [0, 2] Status: 0(PENDING), 1(PENDING), 2(PENDING)
[00:39:49.307] CLIENT_SEND: SEND Data 3
[00:39:49.308] CLIENT_SEND: SEND WINDOW [0, 3] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING)
[00:39:49.308] CLIENT_SEND: SEND Data 4
[00:39:49.309] CLIENT_SEND: SEND WINDOW [0, 4] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING)
[00:39:49.309] CLIENT_SEND: SEND Data 5
[00:39:49.310] CLIENT_SEND: SEND WINDOW [0, 5] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING)
[00:39:49.311] CLIENT_SEND: SEND Data 6
[00:39:49.311] CLIENT_SEND: SEND WINDOW [0, 6] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING)
[00:39:49.312] CLIENT_SEND: SEND Data 7
[00:39:49.312] CLIENT_SEND: SEND WINDOW [0, 7] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING)
[00:39:49.313] CLIENT_SEND: SEND Data 8
[00:39:49.313] CLIENT_SEND: SEND WINDOW [0, 8] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING)
[00:39:49.314] CLIENT_SEND: SEND Data 9
[00:39:49.315] CLIENT_SEND: SEND WINDOW [0, 9] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING)
[00:39:49.315] CLIENT_SEND: SEND Data 10
[00:39:49.316] CLIENT_SEND: SEND WINDOW [0, 10] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING)
[00:39:49.316] CLIENT_SEND: SEND Data 11
[00:39:49.317] CLIENT_SEND: SEND WINDOW [0, 11] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING)
[00:39:49.318] CLIENT_SEND: SEND Data 12
[00:39:49.318] CLIENT_SEND: SEND WINDOW [0, 12] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING),
 12(PENDING)
[00:39:49.319] CLIENT_SEND: SEND Data 13
[00:39:49.319] CLIENT_SEND: SEND WINDOW [0, 13] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING),
 12(PENDING), 13(PENDING)
[00:39:49.320] CLIENT_SEND: SEND Data 14
[00:39:49.321] CLIENT_SEND: SEND WINDOW [0, 14] Status: 0(PENDING), 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING),
 12(PENDING), 13(PENDING), 14(PENDING)
[00:39:49.321] CLIENT_SEND: RECV ACK 0
[00:39:49.322] CLIENT_SEND: Marked Data 0 as ACKed.
[00:39:49.323] CLIENT_SEND: SEND WINDOW base slides to 1
[00:39:49.323] CLIENT_SEND: SEND WINDOW [1, 14] Status: 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING)
```



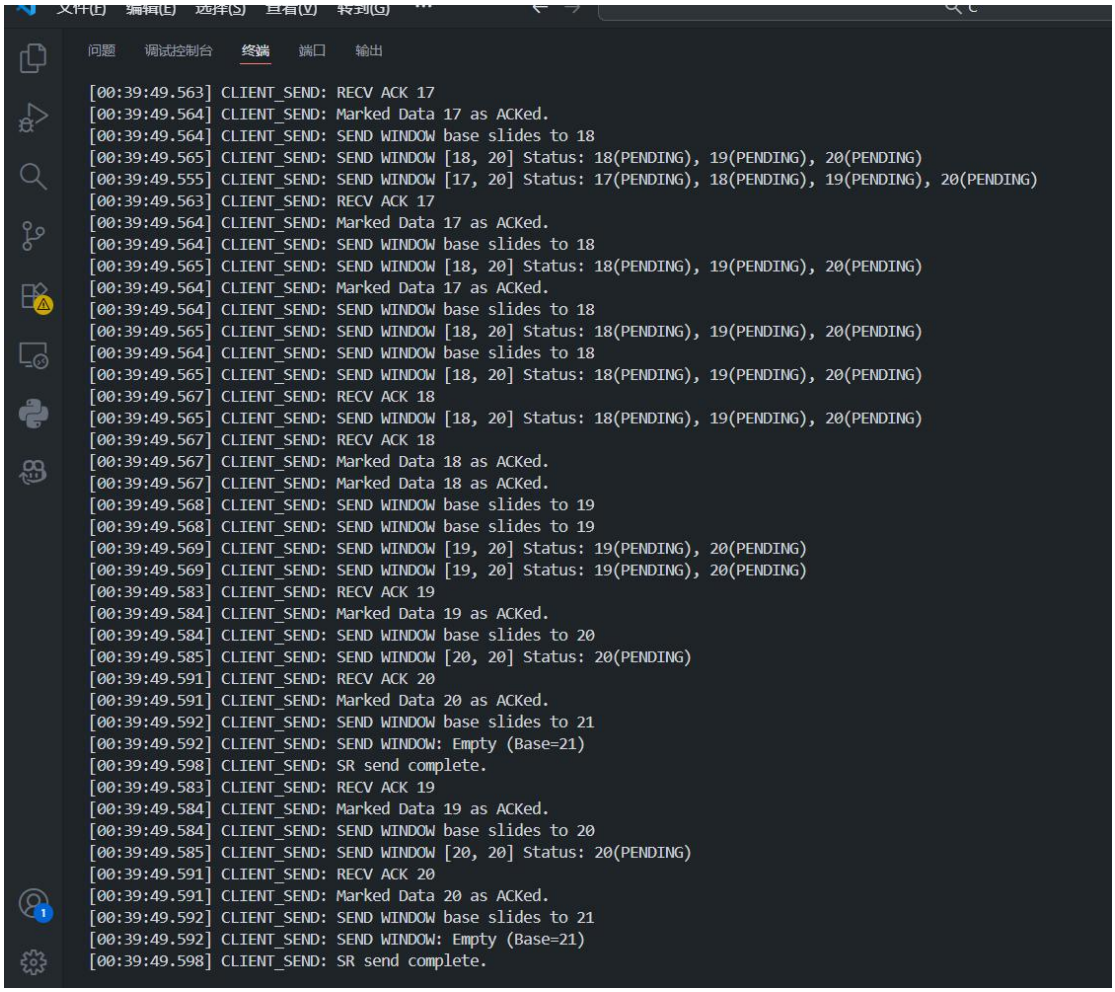
```
[00:39:49.321] CLIENT_SEND: RECV ACK 0
[00:39:49.322] CLIENT_SEND: Marked Data 0 as ACKed.
[00:39:49.323] CLIENT_SEND: SEND WINDOW base slides to 1
[00:39:49.323] CLIENT_SEND: SEND WINDOW [1, 14] Status: 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING)
, 13(PENDING), 14(PENDING)
[00:39:49.337] CLIENT_SEND: SEND Data 15
[00:39:49.338] CLIENT_SEND: SEND WINDOW [1, 15] Status: 1(PENDING), 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING)
, 13(PENDING), 14(PENDING), 15(PENDING)
[00:39:49.338] CLIENT_SEND: RECV ACK 1
[00:39:49.339] CLIENT_SEND: Marked Data 1 as ACKed.
[00:39:49.339] CLIENT_SEND: SEND WINDOW base slides to 2
[00:39:49.340] CLIENT_SEND: SEND WINDOW [2, 15] Status: 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING
), 14(PENDING), 15(PENDING)
[00:39:49.353] CLIENT_SEND: SEND Data 16
[00:39:49.354] CLIENT_SEND: SEND WINDOW [2, 16] Status: 2(PENDING), 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING
), 14(PENDING), 15(PENDING), 16(PENDING)
[00:39:49.355] CLIENT_SEND: RECV ACK 2
[00:39:49.356] CLIENT_SEND: Marked Data 2 as ACKed.
[00:39:49.357] CLIENT_SEND: SEND WINDOW base slides to 3
[00:39:49.357] CLIENT_SEND: SEND WINDOW [3, 16] Status: 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDIN
G), 15(PENDING), 16(PENDING)
[00:39:49.374] CLIENT_SEND: SEND Data 17
[00:39:49.375] CLIENT_SEND: SEND WINDOW [3, 17] Status: 3(PENDING), 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDIN
G), 15(PENDING), 16(PENDING), 17(PENDING)
[00:39:49.376] CLIENT_SEND: RECV ACK 3
[00:39:49.377] CLIENT_SEND: Marked Data 3 as ACKed.
[00:39:49.377] CLIENT_SEND: SEND WINDOW base slides to 4
[00:39:49.378] CLIENT_SEND: SEND WINDOW [4, 17] Status: 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDI
NG), 16(PENDING), 17(PENDING)
[00:39:49.389] CLIENT_SEND: SEND Data 18
[00:39:49.390] CLIENT_SEND: SEND WINDOW [4, 18] Status: 4(PENDING), 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDI
NG), 16(PENDING), 17(PENDING), 18(PENDING)
[00:39:49.391] CLIENT_SEND: RECV ACK 4
[00:39:49.392] CLIENT_SEND: Marked Data 4 as ACKed.
[00:39:49.393] CLIENT_SEND: SEND WINDOW base slides to 5
[00:39:49.393] CLIENT_SEND: SEND WINDOW [5, 18] Status: 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PEND
ING), 17(PENDING), 18(PENDING)
[00:39:49.409] CLIENT_SEND: SEND Data 19
[00:39:49.411] CLIENT_SEND: SEND WINDOW [5, 19] Status: 5(PENDING), 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PEND
ING), 17(PENDING), 18(PENDING), 19(PENDING)
[00:39:49.412] CLIENT_SEND: RECV ACK 5
[00:39:49.413] CLIENT_SEND: Marked Data 5 as ACKed.
```



```
[00:39:49.412] CLIENT_SEND: RECV ACK 5
[00:39:49.413] CLIENT_SEND: Marked Data 5 as ACKed.
[00:39:49.413] CLIENT_SEND: SEND WINDOW base slides to 6
[00:39:49.414] CLIENT_SEND: SEND WINDOW [6, 19] Status: 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PEN
DING), 18(PENDING), 19(PENDING)
[00:39:49.424] CLIENT_SEND: SEND Data 20
[00:39:49.425] CLIENT_SEND: SEND WINDOW [6, 20] Status: 6(PENDING), 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PEN
DING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.425] CLIENT_SEND: RECV ACK 6
[00:39:49.426] CLIENT_SEND: Marked Data 6 as ACKed.
[00:39:49.427] CLIENT_SEND: SEND WINDOW base slides to 7
[00:39:49.427] CLIENT_SEND: SEND WINDOW [7, 20] Status: 7(PENDING), 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PE
NDING), 19(PENDING), 20(PENDING)
[00:39:49.440] CLIENT_SEND: RECV ACK 7
[00:39:49.440] CLIENT_SEND: Marked Data 7 as ACKed.
[00:39:49.441] CLIENT_SEND: SEND WINDOW base slides to 8
[00:39:49.441] CLIENT_SEND: SEND WINDOW [8, 20] Status: 8(PENDING), 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(P
ENDING), 20(PENDING)
[00:39:49.455] CLIENT_SEND: RECV ACK 8
[00:39:49.455] CLIENT_SEND: Marked Data 8 as ACKed.
[00:39:49.456] CLIENT_SEND: SEND WINDOW base slides to 9
[00:39:49.457] CLIENT_SEND: SEND WINDOW [9, 20] Status: 9(PENDING), 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(
PENDING)
[00:39:49.459] CLIENT_SEND: RECV ACK 9
[00:39:49.459] CLIENT_SEND: Marked Data 9 as ACKed.
[00:39:49.460] CLIENT_SEND: SEND WINDOW base slides to 10
[00:39:49.461] CLIENT_SEND: SEND WINDOW [10, 20] Status: 10(PENDING), 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.475] CLIENT_SEND: RECV ACK 10
[00:39:49.476] CLIENT_SEND: Marked Data 10 as ACKed.
[00:39:49.476] CLIENT_SEND: SEND WINDOW base slides to 11
[00:39:49.477] CLIENT_SEND: SEND WINDOW [11, 20] Status: 11(PENDING), 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.486] CLIENT_SEND: RECV ACK 11
[00:39:49.487] CLIENT_SEND: Marked Data 11 as ACKed.
[00:39:49.488] CLIENT_SEND: SEND WINDOW base slides to 12
[00:39:49.488] CLIENT_SEND: SEND WINDOW [12, 20] Status: 12(PENDING), 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.493] CLIENT_SEND: RECV ACK 12
[00:39:49.494] CLIENT_SEND: Marked Data 12 as ACKed.
[00:39:49.495] CLIENT_SEND: SEND WINDOW base slides to 13
[00:39:49.495] CLIENT_SEND: SEND WINDOW [13, 20] Status: 13(PENDING), 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.509] CLIENT_SEND: RECV ACK 13
[00:39:49.510] CLIENT_SEND: Marked Data 13 as ACKed.
[00:39:49.511] CLIENT_SEND: SEND WINDOW base slides to 14
```



```
[00:39:49.510] CLIENT_SEND: Marked Data 13 as ACKed.
[00:39:49.511] CLIENT_SEND: SEND WINDOW base slides to 14
[00:39:49.511] CLIENT_SEND: SEND WINDOW [14, 20] Status: 14(PENDING), 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.521] CLIENT_SEND: RECV ACK 14
[00:39:49.522] CLIENT_SEND: Marked Data 14 as ACKed.
[00:39:49.523] CLIENT_SEND: SEND WINDOW base slides to 15
[00:39:49.523] CLIENT_SEND: SEND WINDOW [15, 20] Status: 15(PENDING), 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.537] CLIENT_SEND: RECV ACK 15
[00:39:49.538] CLIENT_SEND: Marked Data 15 as ACKed.
[00:39:49.539] CLIENT_SEND: SEND WINDOW base slides to 16
[00:39:49.539] CLIENT_SEND: SEND WINDOW [16, 20] Status: 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.553] CLIENT_SEND: RECV ACK 16
[00:39:49.553] CLIENT_SEND: Marked Data 16 as ACKed.
[00:39:49.539] CLIENT_SEND: SEND WINDOW [16, 20] Status: 16(PENDING), 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.553] CLIENT_SEND: RECV ACK 16
[00:39:49.553] CLIENT_SEND: Marked Data 16 as ACKed.
[00:39:49.554] CLIENT_SEND: SEND WINDOW base slides to 17
[00:39:49.555] CLIENT_SEND: SEND WINDOW [17, 20] Status: 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.563] CLIENT_SEND: RECV ACK 17
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.553] CLIENT_SEND: RECV ACK 16
[00:39:49.553] CLIENT_SEND: Marked Data 16 as ACKed.
[00:39:49.554] CLIENT_SEND: SEND WINDOW base slides to 17
[00:39:49.555] CLIENT_SEND: SEND WINDOW [17, 20] Status: 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.563] CLIENT_SEND: RECV ACK 17
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.554] CLIENT_SEND: SEND WINDOW base slides to 17
[00:39:49.555] CLIENT_SEND: SEND WINDOW [17, 20] Status: 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.563] CLIENT_SEND: RECV ACK 17
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.555] CLIENT_SEND: SEND WINDOW [17, 20] Status: 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.563] CLIENT_SEND: RECV ACK 17
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
```



```
问题  调试控制台  终端  端口  输出

[00:39:49.563] CLIENT_SEND: RECV ACK 17
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.555] CLIENT_SEND: SEND WINDOW [17, 20] Status: 17(PENDING), 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.563] CLIENT_SEND: RECV ACK 17
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.564] CLIENT_SEND: Marked Data 17 as ACKed.
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.564] CLIENT_SEND: SEND WINDOW base slides to 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.567] CLIENT_SEND: RECV ACK 18
[00:39:49.565] CLIENT_SEND: SEND WINDOW [18, 20] Status: 18(PENDING), 19(PENDING), 20(PENDING)
[00:39:49.567] CLIENT_SEND: RECV ACK 18
[00:39:49.567] CLIENT_SEND: Marked Data 18 as ACKed.
[00:39:49.567] CLIENT_SEND: Marked Data 18 as ACKed.
[00:39:49.568] CLIENT_SEND: SEND WINDOW base slides to 19
[00:39:49.568] CLIENT_SEND: SEND WINDOW base slides to 19
[00:39:49.569] CLIENT_SEND: SEND WINDOW [19, 20] Status: 19(PENDING), 20(PENDING)
[00:39:49.569] CLIENT_SEND: SEND WINDOW [19, 20] Status: 19(PENDING), 20(PENDING)
[00:39:49.583] CLIENT_SEND: RECV ACK 19
[00:39:49.584] CLIENT_SEND: Marked Data 19 as ACKed.
[00:39:49.584] CLIENT_SEND: SEND WINDOW base slides to 20
[00:39:49.585] CLIENT_SEND: SEND WINDOW [20, 20] Status: 20(PENDING)
[00:39:49.591] CLIENT_SEND: RECV ACK 20
[00:39:49.591] CLIENT_SEND: Marked Data 20 as ACKed.
[00:39:49.592] CLIENT_SEND: SEND WINDOW base slides to 21
[00:39:49.592] CLIENT_SEND: SEND WINDOW: Empty (Base=21)
[00:39:49.598] CLIENT_SEND: SR send complete.
[00:39:49.583] CLIENT_SEND: RECV ACK 19
[00:39:49.584] CLIENT_SEND: Marked Data 19 as ACKed.
[00:39:49.584] CLIENT_SEND: SEND WINDOW base slides to 20
[00:39:49.585] CLIENT_SEND: SEND WINDOW [20, 20] Status: 20(PENDING)
[00:39:49.591] CLIENT_SEND: RECV ACK 20
[00:39:49.591] CLIENT_SEND: Marked Data 20 as ACKed.
[00:39:49.592] CLIENT_SEND: SEND WINDOW base slides to 21
[00:39:49.592] CLIENT_SEND: SEND WINDOW: Empty (Base=21)
[00:39:49.598] CLIENT_SEND: SR send complete.
```

SR协议服务器端

```
[00:39:23.066] SERVER: SR Server running, waiting for commands on port 8888
[00:39:23.068] SERVER: Waiting for new command...
[00:39:30.721] SERVER: Received command: '-quit' Filename: ''
[00:39:30.722] SERVER: Waiting for new command...
[00:39:49.298] SERVER: Received command: 'upload' Filename: 'text2.txt'
Starting SR receive...
[00:39:49.300] SERVER_RECV: Entering SR receive state for file: server_sr_text2.txt
[00:39:49.301] SERVER_RECV: Data Loss (X): 0.000000, ACK Loss (Y): 0.000000
[00:39:49.302] SERVER_RECV: RECV WINDOW [0, 14] Status: 0(EXPECTED), 1(EMPTY), 2(EMPTY), 3(EMPTY), 4(EMPTY), 5(EMPTY), 6(EMPT
Y), 7(EMPTY), 8(EMPTY), 9(EMPTY), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY)
Debug: recvLen = 1028
[00:39:49.304] SERVER_RECV: RECV Frame 0
[00:39:49.304] SERVER_RECV: DELIVER Data 0
[00:39:49.305] SERVER_RECV: SEND ACK 0
[00:39:49.306] SERVER_RECV: RECV WINDOW [1, 15] Status: 1(EXPECTED), 2(EMPTY), 3(EMPTY), 4(EMPTY), 5(EMPTY), 6(EMPTY), 7(EMPT
Y), 8(EMPTY), 9(EMPTY), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY)
Debug: recvLen = 1028
[00:39:49.307] SERVER_RECV: RECV Frame 1
[00:39:49.308] SERVER_RECV: DELIVER Data 1
[00:39:49.309] SERVER_RECV: SEND ACK 1
[00:39:49.310] SERVER_RECV: RECV WINDOW [2, 16] Status: 2(EXPECTED), 3(EMPTY), 4(EMPTY), 5(EMPTY), 6(EMPTY), 7(EMPTY), 8(EMPT
Y), 9(EMPTY), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY)
Debug: recvLen = 1028
[00:39:49.311] SERVER_RECV: RECV Frame 2
[00:39:49.311] SERVER_RECV: DELIVER Data 2
[00:39:49.312] SERVER_RECV: SEND ACK 2
[00:39:49.313] SERVER_RECV: RECV WINDOW [3, 17] Status: 3(EXPECTED), 4(EMPTY), 5(EMPTY), 6(EMPTY), 7(EMPTY), 8(EMPTY), 9(EMPT
Y), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY)
Debug: recvLen = 1028
[00:39:49.314] SERVER_RECV: RECV Frame 3
[00:39:49.314] SERVER_RECV: DELIVER Data 3
[00:39:49.315] SERVER_RECV: SEND ACK 3
[00:39:49.316] SERVER_RECV: RECV WINDOW [4, 18] Status: 4(EXPECTED), 5(EMPTY), 6(EMPTY), 7(EMPTY), 8(EMPTY), 9(EMPTY), 10(EMP
TY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY)
Debug: recvLen = 1028
[00:39:49.317] SERVER_RECV: RECV Frame 4
[00:39:49.317] SERVER_RECV: DELIVER Data 4
[00:39:49.318] SERVER_RECV: SEND ACK 4
[00:39:49.319] SERVER_RECV: RECV WINDOW [5, 19] Status: 5(EXPECTED), 6(EMPTY), 7(EMPTY), 8(EMPTY), 9(EMPTY), 10(EMPTY), 11(EM
PTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY)
Debug: recvLen = 1028
[00:39:49.320] SERVER_RECV: RECV Frame 5
[00:39:49.321] SERVER_RECV: DELIVER Data 5
[00:39:49.322] SERVER_RECV: SEND ACK 5
[00:39:49.322] SERVER_RECV: RECV WINDOW [6, 20] Status: 6(EXPECTED), 7(EMPTY), 8(EMPTY), 9(EMPTY), 10(EMPTY), 11(EMPTY), 12(E
MPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY)
Debug: recvLen = 1028
[00:39:49.323] SERVER_RECV: RECV Frame 6
[00:39:49.324] SERVER_RECV: DELIVER Data 6
[00:39:49.325] SERVER_RECV: SEND ACK 6
[00:39:49.325] SERVER_RECV: RECV WINDOW [7, 21] Status: 7(EXPECTED), 8(EMPTY), 9(EMPTY), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(
EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY)
Debug: recvLen = 1028
[00:39:49.326] SERVER_RECV: RECV Frame 7
[00:39:49.327] SERVER_RECV: DELIVER Data 7
[00:39:49.328] SERVER_RECV: SEND ACK 7
[00:39:49.328] SERVER_RECV: RECV WINDOW [8, 22] Status: 8(EXPECTED), 9(EMPTY), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14
(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY)
Debug: recvLen = 1028
[00:39:49.329] SERVER_RECV: RECV Frame 8
[00:39:49.330] SERVER_RECV: DELIVER Data 8
[00:39:49.331] SERVER_RECV: SEND ACK 8
[00:39:49.331] SERVER_RECV: RECV WINDOW [9, 23] Status: 9(EXPECTED), 10(EMPTY), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 1
5(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY)
Debug: recvLen = 1028
[00:39:49.332] SERVER_RECV: RECV Frame 9
[00:39:49.333] SERVER_RECV: DELIVER Data 9
[00:39:49.334] SERVER_RECV: SEND ACK 9
[00:39:49.334] SERVER_RECV: RECV WINDOW [10, 24] Status: 10(EXPECTED), 11(EMPTY), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY),
 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY)
Debug: recvLen = 1028
[00:39:49.335] SERVER_RECV: RECV Frame 10
[00:39:49.336] SERVER_RECV: DELIVER Data 10
[00:39:49.336] SERVER_RECV: SEND ACK 10
[00:39:49.337] SERVER_RECV: RECV WINDOW [11, 25] Status: 11(EXPECTED), 12(EMPTY), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY),
 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY)
Debug: recvLen = 1028
[00:39:49.339] SERVER_RECV: RECV Frame 11
[00:39:49.339] SERVER_RECV: DELIVER Data 11
```

```
[00:39:49.339] SERVER_RECV: RECV Frame 11
[00:39:49.339] SERVER_RECV: DELIVER Data 11
[00:39:49.340] SERVER_RECV: SEND ACK 11
[00:39:49.341] SERVER_RECV: RECV WINDOW [12, 26] Status: 12(EXPECTED), 13(EMPTY), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY),
 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY)
Debug: recvLen = 1028
[00:39:49.342] SERVER_RECV: RECV Frame 12
[00:39:49.343] SERVER_RECV: DELIVER Data 12
[00:39:49.343] SERVER_RECV: SEND ACK 12
[00:39:49.344] SERVER_RECV: RECV WINDOW [13, 27] Status: 13(EXPECTED), 14(EMPTY), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY),
 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY)
Debug: recvLen = 1028
[00:39:49.345] SERVER_RECV: RECV Frame 13
[00:39:49.345] SERVER_RECV: DELIVER Data 13
[00:39:49.346] SERVER_RECV: SEND ACK 13
[00:39:49.347] SERVER_RECV: RECV WINDOW [14, 28] Status: 14(EXPECTED), 15(EMPTY), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY),
 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY)
Debug: recvLen = 1028
[00:39:49.348] SERVER_RECV: RECV Frame 14
[00:39:49.349] SERVER_RECV: DELIVER Data 14
[00:39:49.349] SERVER_RECV: SEND ACK 14
[00:39:49.350] SERVER_RECV: RECV WINDOW [15, 29] Status: 15(EXPECTED), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY),
 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY)
Debug: recvLen = 1028
[00:39:49.351] SERVER_RECV: RECV Frame 15
[00:39:49.351] SERVER_RECV: DELIVER Data 15
[00:39:49.352] SERVER_RECV: SEND ACK 15
[00:39:49.353] SERVER_RECV: RECV WINDOW [16, 30] Status: 16(EXPECTED), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY),
 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY)
Debug: recvLen = 1028
[00:39:49.354] SERVER_RECV: RECV Frame 16
[00:39:49.355] SERVER_RECV: DELIVER Data 16
[00:39:49.356] SERVER_RECV: SEND ACK 16
[00:39:49.357] SERVER_RECV: RECV WINDOW [17, 31] Status: 17(EXPECTED), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY),
 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY)
Debug: recvLen = 1028
[00:39:49.375] SERVER_RECV: RECV Frame 17
[00:39:49.376] SERVER_RECV: DELIVER Data 17
[00:39:49.377] SERVER_RECV: SEND ACK 17
[00:39:49.378] SERVER_RECV: RECV WINDOW [18, 32] Status: 18(EXPECTED), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY),
 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY), 32(EMPTY)
```

```
[00:39:49.350] SERVER_RECV: RECV WINDOW [15, 29] Status: 15(EXPECTED), 16(EMPTY), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY),
 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY)
Debug: recvLen = 1028
[00:39:49.351] SERVER_RECV: RECV Frame 15
[00:39:49.351] SERVER_RECV: DELIVER Data 15
[00:39:49.352] SERVER_RECV: SEND ACK 15
[00:39:49.353] SERVER_RECV: RECV WINDOW [16, 30] Status: 16(EXPECTED), 17(EMPTY), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY),
 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY)
Debug: recvLen = 1028
[00:39:49.354] SERVER_RECV: RECV Frame 16
[00:39:49.355] SERVER_RECV: DELIVER Data 16
[00:39:49.356] SERVER_RECV: SEND ACK 16
[00:39:49.357] SERVER_RECV: RECV WINDOW [17, 31] Status: 17(EXPECTED), 18(EMPTY), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY),
 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY)
Debug: recvLen = 1028
[00:39:49.375] SERVER_RECV: RECV Frame 17
[00:39:49.376] SERVER_RECV: DELIVER Data 17
[00:39:49.377] SERVER_RECV: SEND ACK 17
[00:39:49.378] SERVER_RECV: RECV WINDOW [18, 32] Status: 18(EXPECTED), 19(EMPTY), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY),
 24(EMPTY), 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY), 32(EMPTY)
Debug: recvLen = 1028
[00:39:49.391] SERVER_RECV: RECV Frame 18
[00:39:49.392] SERVER_RECV: DELIVER Data 18
[00:39:49.392] SERVER_RECV: SEND ACK 18
[00:39:49.393] SERVER_RECV: RECV WINDOW [19, 33] Status: 19(EXPECTED), 20(EMPTY), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY),
 25(EMPTY), 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY), 32(EMPTY), 33(EMPTY)
Debug: recvLen = 1028
[00:39:49.411] SERVER_RECV: RECV Frame 19
[00:39:49.412] SERVER_RECV: DELIVER Data 19
[00:39:49.413] SERVER_RECV: SEND ACK 19
[00:39:49.414] SERVER_RECV: RECV WINDOW [20, 34] Status: 20(EXPECTED), 21(EMPTY), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY),
 26(EMPTY), 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY), 32(EMPTY), 33(EMPTY), 34(EMPTY)
Debug: recvLen = 1028
[00:39:49.425] SERVER_RECV: RECV Frame 20
[00:39:49.426] SERVER_RECV: DELIVER Data 20
[00:39:49.427] SERVER_RECV: SEND ACK 20
[00:39:49.427] SERVER_RECV: RECV WINDOW [21, 35] Status: 21(EXPECTED), 22(EMPTY), 23(EMPTY), 24(EMPTY), 25(EMPTY), 26(EMPTY),
 27(EMPTY), 28(EMPTY), 29(EMPTY), 30(EMPTY), 31(EMPTY), 32(EMPTY), 33(EMPTY), 34(EMPTY), 35(EMPTY)
[00:39:49.428] SERVER_RECV: EOF received and buffer empty. Transfer complete.
[00:39:49.429] SERVER: Waiting for new command...
[00:39:55.438] SERVER: Waiting for new command
```

问题讨论：

本次实验给我最大的启发就是对来自客户命令和数据帧的响应问题。在一开始时，直接粗暴使用 recvfrom()提供的计时功能进行对是否超时进行监控，但是发现容易因此错过命令或者数据帧。于是想到短时间多次循环、单独设置计时器计算是否超时、进入接收状态后改变recvfrom()函数的超时时间，这些方法来解决粗暴地只使用 recvfrom()函数提供的功能来实现超市监控引起的忽略指令的问题。

心得体会：

本次实验的核心是基于不可靠的 UDP 协议，构建出两种可靠的数据传输机制：回退 N 步和选择重传。通过实现这两种协议，我们深入探讨了可靠数据传输协议的核心机制、滑动窗口原理及其在不同策略下的效率差异。实验的主要内容可以概括为对协议栈核心功能的完整实现，包括数据帧的封装、序列号管理、超时重传、错误检测以及窗口动态管理。

在协议实现中，我们首要关注的是数据帧的结构化和错误检测。数据帧被扩展为包 1 字节序列号、2 字节 16 位互联网校验和、以及 1024 字节数据载荷，这确保了每个数据单元都带有可识别的身份和可验证的完整性信息。校验和逻辑的正确实现成为本次实验中最具挑战性的环节之一。最初，由于平台默认的 char 有符号性以及对网络字节序的隐式处理不足，导致发送方和接收方对同一字节流计算出的校验和结果不一致，引发了持续的 DISCARD 错误。通过引入 uint8_t 指针和强制位移操作来确保字节到 16 位字的转换始终遵循网络字节序标准，我们最终解决了这一底层问题，证明了在网络编程中，数据类型和字节序处理的严格性至关重要。

在功能实现方面，GBN 协议展现了其简洁性。发送方仅需一个计时器来监控窗口基序号，一旦超时便执行回退 N 步，重传整个未确认窗口内的数据包，这在 ACK 丢失率低的环境下具有效率优势。相比之下，SR 协议则要求更精细的状态管理：发送方为窗口内每个未确认数据包设置独立计时器，只重传超时的数据包；而接收方则实现了乱序接收缓冲区，能接收并缓存窗口内的任意数据帧，待基序号的包到达后，再按序交付连续缓存的数据。通过动态输出的窗口状态日志，我们可以看到窗口滑动时连续交付缓存数据帧的整个过程，成功验证了 SR 协议的选择重传和乱序交付特性，这是其区别于 GBN 的关键优势。